

Laporan Tugas Besar
IF2224 Teori Bahasa Formal dan Automata
Milestone 2 - Syntax Analysis



Disusun oleh:

13524071 Kalyca Nathania B Manullang

13524073 Keisha Rizka Syofyani

13524087 Muhammad Fakhry Zaki

13524109 Helena Kristela Sarhawa

Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Landasan Teori

2.1 Analisis Sintaks dan Peran Parser

Analisis sintaks (*syntax analysis*) merupakan tahap kedua dalam proses kompilasi setelah analisis leksikal. Pada tahap ini, deretan token yang dihasilkan oleh lexer akan diperiksa apakah telah sesuai dengan aturan tata bahasa (*grammar*) dari bahasa pemrograman yang digunakan. Proses ini dilakukan oleh komponen yang disebut parser. Parser bertugas untuk memastikan bahwa struktur program yang ditulis mengikuti aturan sintaks yang benar sehingga program dapat diproses lebih lanjut oleh tahap berikutnya dalam kompilasi.

Parser tidak lagi bekerja langsung pada *source code*, melainkan pada *list of tokens* hasil dari lexer. Maka dari itu, parser berperan sebagai penghubung antara hasil analisis leksikal dengan analisis semantik. Selain melakukan validasi terhadap struktur token, parser juga berfungsi untuk mendeteksi kesalahan sintaks (*syntax error*). Kesalahan ini terjadi apabila urutan token tidak sesuai dengan aturan *grammar* yang telah ditentukan. Oleh karena itu, analisis sintaks memiliki peran penting dalam memastikan bahwa struktur program sudah benar sebelum diproses lebih lanjut.

2.2 Konsep Grammar dan Aturan Produksi

Dalam teori bahasa formal, *grammar* merupakan sekumpulan aturan yang digunakan untuk mendefinisikan struktur suatu bahasa. *Grammar* terdiri dari simbol terminal, simbol nonterminal, simbol awal (*start symbol*), serta aturan produksi. Simbol terminal adalah simbol yang muncul langsung dalam input, sedangkan simbol nonterminal merepresentasikan struktur abstrak dalam bahasa.

Aturan produksi mendefinisikan bagaimana suatu simbol nonterminal dapat diturunkan menjadi kombinasi simbol lain. Aturan ini biasanya dinyatakan dalam bentuk formal seperti Backus-Naur Form (BNF) yang memudahkan dalam mendeskripsikan struktur bahasa secara sistematis.

2.3 Parse Tree sebagai Representasi Sintaks

Parse tree merupakan representasi pohon yang menggambarkan struktur sintaks dari suatu program berdasarkan aturan *grammar*. Setiap node dalam parse tree merepresentasikan simbol dalam *grammar*, baik simbol terminal maupun nonterminal. Struktur ini menunjukkan bagaimana suatu program diturunkan dari simbol awal melalui penerapan aturan produksi secara bertahap. Parse tree bersifat lengkap karena memuat seluruh detail proses derivasi, termasuk semua simbol yang terlibat. Maka dari itu, parse tree mampu memberikan gambaran yang jelas mengenai struktur hierarkis suatu program.

2.4 Perbedaan Parse Tree dan Abstract Syntax Tree (AST)

Parse tree dan Abstract Syntax Tree (AST) merupakan dua bentuk representasi struktur program yang sering digunakan dalam proses kompilasi. Parse tree merepresentasikan struktur sintaks secara lengkap sesuai dengan aturan *grammar*, termasuk semua simbol terminal dan nonterminal. Sebaliknya, AST merupakan bentuk yang lebih sederhana karena hanya mempertahankan elemen-elemen penting yang diperlukan untuk analisis semantik dan pembangkitan kode. Perbedaan utama antara keduanya terletak pada tingkat detail yang disimpan. Parse tree menyimpan seluruh informasi derivasi, sedangkan AST menghilangkan elemen-elemen yang tidak diperlukan, seperti tanda kurung atau simbol tertentu yang tidak mempengaruhi makna program.

2.5 Metode Parsing Top-Down dan Recursive Descent

Parsing dapat dilakukan dengan berbagai metode, salah satunya yakni top-down parsing. Metode ini bekerja dengan cara memulainya dari simbol awal (*start symbol*) dan mencoba menurunkan input hingga sesuai dengan token yang diberikan. Pendekatan ini mengikuti struktur *grammar* secara langsung sehingga relatif mudah dipahami.

Salah satu implementasi dari top-down parsing adalah Recursive Descent Parser. Metode ini mengimplementasikan setiap aturan produksi dalam *grammar* sebagai fungsi rekursif. Parser akan memanggil fungsi-fungsi tersebut sesuai dengan struktur *grammar* untuk memverifikasi apakah input sesuai dengan aturan yang ada. Recursive descent parser memiliki keunggulan dalam kemudahan implementasi dan fleksibilitas. Namun, metode ini memiliki keterbatasan

seperti tidak dapat menangani *grammar* dengan rekursi kiri secara langsung sehingga *grammar* perlu disesuaikan terlebih dahulu agar dapat digunakan dengan metode ini.

2.6 Konsep Derivasi dalam Parsing

Derivasi merupakan proses penerapan aturan produksi secara bertahap untuk menghasilkan string dari simbol awal. Dalam parsing, derivasi digunakan untuk menjelaskan bagaimana suatu input dapat dihasilkan dari *grammar* yang ada. Terdapat dua jenis derivasi yang umum digunakan yakni leftmost derivation dan rightmost derivation. Leftmost derivation selalu mengembangkan simbol nonterminal paling kiri, sedangkan rightmost derivation mengembangkan simbol nonterminal paling kanan terlebih dahulu.

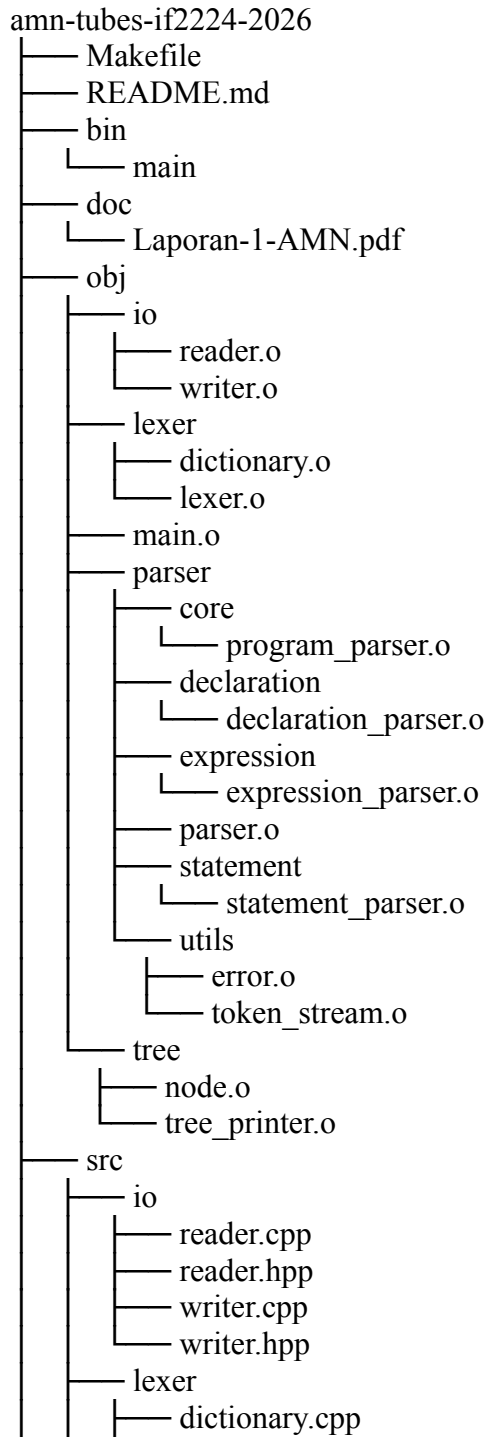
2.7 Penanganan Error pada Analisis Sintaks

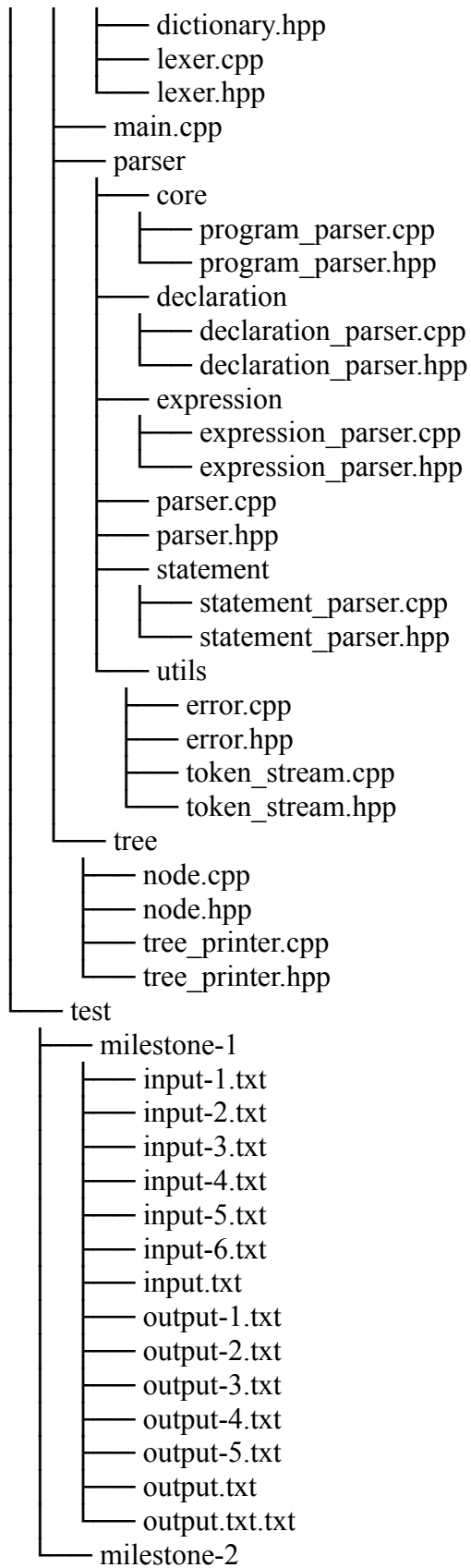
Selama proses analisis sintaks, terdapat kemungkinan munculnya kesalahan yang disebut sebagai *syntax error*. Kesalahan ini terjadi ketika urutan token tidak sesuai dengan aturan *grammar* yang telah ditentukan, seperti struktur yang tidak lengkap atau penulisan yang tidak sesuai. Parser harus mampu mendeteksi kesalahan tersebut dan memberikan informasi yang jelas terkait lokasi dan jenis kesalahan. Dalam beberapa implementasi, parser juga dilengkapi dengan mekanisme *error recovery* agar proses parsing dapat tetap dilanjutkan meskipun terjadi kesalahan.

Perancangan & Implementasi

a. Struktur File Program

Berikut adalah struktur folder dan file program pada tugas Syntax Analysis ini.





- input-1.txt
- input-2.txt
- input-3.txt
- input-4.txt
- input-5.txt
- input-6.txt
- input-pretest.txt
- output-6-lexer.txt
- output-1-lexer.txt
- output-1-parser.txt
- output-2-lexer.txt
- output-2-parser.txt
- output-3-lexer.txt
- output-4-lexer.txt
- output-4-parser.txt
- output-5-lexer.txt
- output-5-parser.txt
- output-6-lexer.txt
- output-6-parser.txt
- output-pretest.txt

Implementasi parser pada milestone 2 ini dikembangkan dengan menggunakan bahasa pemrograman C++ yang menggunakan pendekatan OOP. Struktur folder dan pendekatan integrasi pada Milestone 2 ini adalah penyempurnaan dari Milestone 1 yang masih prosedural dan kurang menerapkan konsep modular. Program pada Milestone 2 ini disusun dengan pendekatan modular untuk meningkatkan keterbacaan kode, kemudahan pengembangan, serta meminimalkan konflik dalam kerja tim. Struktur folder dirancang dengan memisahkan setiap komponen utama berdasarkan tanggung jawabnya masing-masing pada kelas-kelas utama, terutama mengikuti pembagian grammar pada bahasa Arion. Program ini sudah menyesuaikan untuk fungsionalitas input/output, modul setiap milestone yang terpisah, hingga running program yang sudah lebih efisien. Dengan pendekatan ini, setiap bagian parser dapat dikembangkan secara independen tanpa mengganggu bagian lain, sekaligus mempermudah proses integrasi secara keseluruhan.

Secara umum, struktur program dibagi ke dalam beberapa direktori utama, yaitu lexer, parser, tree, dan io. Direktori lexer merupakan hasil pengembangan pada milestone sebelumnya yang bertugas melakukan analisis leksikal dan menghasilkan token sebagai input bagi parser. Direktori parser menjadi inti dari milestone ini, yang kemudian dibagi lagi menjadi beberapa submodul seperti core, declaration, statement, expression, dan utils. Pembagian ini dilakukan

berdasarkan kategori grammar sehingga setiap submodul hanya menangani satu jenis struktur sintaks tertentu. Sebagai contoh, core menangani alur utama parsing program, declaration menangani seluruh bentuk deklarasi, statement berfokus pada struktur kontrol dan pernyataan, serta expression menangani evaluasi ekspresi. Sementara itu, utils menyediakan komponen pendukung seperti pengelolaan token (TokenStream) dan mekanisme penanganan error. Selain itu, direktori tree digunakan untuk merepresentasikan hasil parsing dalam bentuk parse tree. Pada bagian ini, struktur node didefinisikan untuk membangun hierarki sintaks, serta disediakan fungsi untuk mencetak tree dalam format terstruktur sesuai spesifikasi. Direktori io berfungsi sebagai pendukung untuk membaca input dari file dan menuliskan hasil parse tree ke dalam file output, sehingga memisahkan logika input/output dari logika parsing utama.

b. Implementasi Utama Parser (Core)

Main program parser

```
#include "lexer/lexer.hpp"
#include "io/reader.hpp"
#include "io/writer.hpp"
#include "parser/parser.hpp"
#include "parser/utils/error.hpp"

// std::vector<std::string> token_list;
// Node* root;

int main () {

    /* ===== LEXER ===== */
    INIT_DICTIONARY();
    INPUT_FILE();
    READ_ALL_FILE();
    PRINT_TOKEN_LIST();
    SAVE_TOKEN_LIST();

    /* ===== PARSER ===== */
    try{
        Parser parser(token_list);
        Node* root = parser.parse();

        PRINT_PARSE_TREE (root, 0);

        SAVE_PARSE_TREE (root, 0);

        delete root;
    } catch (const SyntaxError& e) {
        std::cerr << e.what() <<std::endl;
    }
```



```

        // return 1;
    }

    return 0;
}

```

Header file parser

```

#ifndef PARSER_HPP
#define PARSER_HPP

#include <vector>
#include <string>
#include "tree/node.hpp"
#include "parser/utils/token_stream.hpp"
#include "parser/core/program_parser.hpp"

class Parser {
private:
    TokenStream stream;

public:
    // Constructor menerima token list dari lexer
    Parser(const std::vector<std::string>& tokens);

    // Jalankan parsing, return root parse tree
    // Throws SyntaxError jika ada kesalahan syntax
    Node* parse();
};

#endif

```

Implementasi parser

```

#include "parser.hpp"

Parser::Parser(const std::vector<std::string>& tokens) : stream(tokens) {}

// Jalankan parsing, return root parse tree
// Throws SyntaxError jika ada kesalahan syntax
Node* Parser::parse() {
    return parseProgram(stream);
}

```

Modul program parser

```

#include "parser/core/program_parser.hpp"
#include "parser/utils/error.hpp"
#include "parser/statement/statement_parser.hpp"

Node* parseProgram(TokenStream& ts) {

```

```

Node* node = new Node("<program>");

// Parse program header
node->addChild(parseProgramHeader(ts));

// Parse declaration part
node->addChild(parseDeclarationPartCoordinator(ts));

// Parse compound statement
node->addChild(parseCompoundStatementCoordinator(ts));

// Harus diakhiri dengan period
if (!ts.check("period")) {
    throwSyntaxError(ts.current(), "period", ts.getIndex());
}
node->addChild(new Node(ts.current())); // terminal node "period"
ts.advance();

return node;
}

// <program-header> → programsy ident semicolon
Node* parseProgramHeader(TokenStream& ts) {
    Node* node = new Node("<program-header>");

    // Expect 'programsy'
    if (!ts.check("programsy")) {
        throwSyntaxError(ts.current(), "programsy", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();

    // Expect minimal 1 identifier
    if (!ts.check("ident")) {
        throwSyntaxError(ts.current(), "ident", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();

    while (ts.check("ident") || ts.check("intcon")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
    }

    // Expect semicolon
    if (!ts.check("semicolon")) {
        throwSyntaxError(ts.current(), "semicolon", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();

    return node;
}

```

```

// koordinasi declaration : const/type/var/subprogram
Node* parseDeclarationPartCoordinator(TokenStream& ts) {
    Node* node = new Node("<declaration-part>");

    while (ts.check("constsy") || ts.check("typesy") || ts.check("varsy")
|| ts.check("proceduresy") || ts.check("functionsy")) {
        // Parse const declaration
        if(ts.check("constsy")) node->addChild(parseConstDeclaration(ts));

        // Parse type declaration
        else if(ts.check("typesy"))
node->addChild(parseTypeDeclaration(ts));

        // Parse var declaration
        else if(ts.check("varsy"))
node->addChild(parseVarDeclaration(ts));

        // Parse subprogram declaration
        else node->addChild(parseSubprogramDeclaration(ts));
    }

    return node;
}

// koordinasi statement : list/if/while/..
Node* parseCompoundStatementCoordinator(TokenStream& ts) {
    return parseCompoundStatement(ts);
}

```

Parser utama berperan sebagai pengatur alur proses analisis sintaks secara keseluruhan. Implementasi bagian ini mencakup integrasi antara hasil lexical analysis (token) dengan modul-modul parser lainnya untuk menghasilkan parse tree yang sesuai dengan grammar bahasa Arion. Komponen utama pada bagian ini terdiri dari main.cpp, parser.cpp, dan program_parser.cpp, yang bekerja secara berurutan dalam menjalankan proses parsing.

Proses dimulai dari fungsi main() yang bertugas sebagai entry point program. Pada tahap awal, program menjalankan proses lexical analysis dengan memanggil fungsi-fungsi seperti INIT_DICTIONARY(), INPUT_FILE(), READ_ALL_FILE(), hingga PRINT_TOKEN_LIST() dan SAVE_TOKEN_LIST(). Tahap ini menghasilkan daftar token yang kemudian menjadi input bagi parser. Setelah proses lexer selesai, parser diinisialisasi dengan memberikan token_list ke dalam objek Parser. Proses parsing kemudian dijalankan melalui pemanggilan fungsi parse(), yang akan menghasilkan root dari parse tree. Hasil parse tree tersebut selanjutnya ditampilkan ke terminal menggunakan PRINT_PARSE_TREE() dan juga disimpan ke dalam file melalui

SAVE_PARSE_TREE(). Mekanisme penanganan error dilakukan menggunakan blok try-catch, di mana kesalahan sintaks akan ditangani melalui exception `SyntaxError` dan ditampilkan dalam bentuk pesan yang informatif.

Pada file `parser.cpp`, kelas `Parser` berfungsi sebagai penghubung antara token stream dengan fungsi parsing utama. Konstruktor `Parser` menerima daftar token dan menginisialisasi objek `TokenStream` yang digunakan selama proses parsing. Fungsi `parse()` pada kelas ini menjadi titik awal parsing dengan memanggil fungsi `parseProgram()`, yang merupakan implementasi dari non-terminal `<program>` dalam grammar.

Implementasi utama dari grammar `<program>` terdapat pada file `program_parser.cpp`. Fungsi `parseProgram()` bertanggung jawab untuk membangun struktur parse tree dari keseluruhan program. Sesuai dengan aturan produksi, fungsi ini memanggil tiga komponen utama secara berurutan, yaitu `parseProgramHeader()`, `parseDeclarationPartCoordinator()`, dan `parseCompoundStatementCoordinator()`. Setelah ketiga bagian tersebut diproses, parser melakukan validasi terhadap token terakhir yang harus berupa period. Jika token yang ditemukan tidak sesuai, maka akan dihasilkan error menggunakan fungsi `throwSyntaxError()`.

Fungsi `parseProgramHeader()` mengimplementasikan aturan produksi `<program-header>` → `programsy ident semicolon`. Pada implementasinya, fungsi ini tidak hanya memeriksa keberadaan satu identifier, tetapi juga mengakomodasi kemungkinan adanya beberapa token tambahan seperti identifier atau integer constant setelah nama program, sebelum diakhiri dengan semicolon. Hal ini dilakukan untuk meningkatkan fleksibilitas parsing terhadap variasi input. Selanjutnya, fungsi `parseDeclarationPartCoordinator()` bertugas untuk mengatur parsing bagian deklarasi. Fungsi ini menggunakan perulangan untuk memproses berbagai jenis deklarasi seperti `const`, `type`, `var`, serta subprogram (`procedure` dan `function`). Setiap jenis deklarasi akan diarahkan ke fungsi parser yang sesuai, sehingga menjaga modularitas dan keterpisahan tanggung jawab antar komponen. Terakhir, fungsi `parseCompoundStatementCoordinator()` bertindak sebagai penghubung ke parser statement, khususnya untuk memproses blok utama program yang dibatasi oleh `begin` dan `end`. Fungsi ini mendelegasikan proses parsing ke `parseCompoundStatement()` yang berada pada modul statement parser.

c. Implementasi Declaration Parser

Modul declaration parser

```
#include "declaration_parser.hpp"
#include "parser/utils/error.hpp"
#include "parser/statement/statement_parser.hpp"

static std::string peekTypeTrimmed(TokenStream& ts) {
    std::string t = ts.peekType();
    while (!t.empty() && t.back() == ' ') {
        t.pop_back();
    }
    return t;
}

// cek apakah token saat ini bisa menjadi awal <constant>
Node* parseConstant(TokenStream& ts) {
    Node* node = new Node("<constant>");
    std::string t = ts.currentType();
    if (t == "charcon") {
        node->addChild(new Node(ts.current()));
        ts.advance();
        return node;
    }
    if (t == "string") {
        node->addChild(new Node(ts.current()));
        ts.advance();
        return node;
    }
    if (t == "plus" || t == "minus") {
        node->addChild(new Node(ts.current()));
        ts.advance();
        t = ts.currentType();
    }
    if (t == "ident" || t == "intcon" || t == "realcon") {
        node->addChild(new Node(ts.current()));
        ts.advance();
        return node;
    }
    throwSyntaxError(ts.current(), "constant (charcon | string | ident | intcon | realcon)", ts.getIndex());
    return nullptr;
}

Node* parseIdentifierList(TokenStream& ts) {
    Node* node = new Node("<identifier-list>");
    if (!ts.check("ident")) {
        throwSyntaxError(ts.current(), "ident", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    while (ts.check("comma")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
    }
}
```

```

        if (!ts.check("ident")) {
            throwSyntaxError(ts.current(), "ident after comma",
ts.getIndex());
        }
        node->addChild(new Node(ts.current()));
        ts.advance();
    }
    return node;
}

Node* parseRange(TokenStream& ts) {
    Node* node = new Node("<range>");
    node->addChild(parseConstant(ts));
    if (!ts.check("period")) {
        throwSyntaxError(ts.current(), "period (titik pertama range)",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("period")) {
        throwSyntaxError(ts.current(), "period (titik kedua range)",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    node->addChild(parseConstant(ts));
    return node;
}

Node* parseEnumerated(TokenStream& ts) {
    Node* node = new Node("<enumerated>");
    if (!ts.check("lparent")) {
        throwSyntaxError(ts.current(), "lparent", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("ident")) {
        throwSyntaxError(ts.current(), "ident di dalam enumerated",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    while (ts.check("comma")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
        if (!ts.check("ident")) {
            throwSyntaxError(ts.current(), "ident setelah comma di
enumerated", ts.getIndex());
        }
        node->addChild(new Node(ts.current()));
        ts.advance();
    }
    if (!ts.check("rparent")) {

```

```

        throwSyntaxError(ts.current(), "rparent penutup enumerated",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    return node;
}

Node* parseArrayType(TokenStream& ts) {
    Node* node = new Node("<array-type>");
    if (!ts.check("arraysy")) {
        throwSyntaxError(ts.current(), "arraysy", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("lbrack")) {
        throwSyntaxError(ts.current(), "lbrack", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    std::string t = ts.currentType();
    bool isRange = false;
    if (t == "plus" || t == "minus") {
        isRange = true;
    } else if (t == "intcon" || t == "realcon" || t == "charcon" || t ==
"string") {
        isRange = true;
    } else if (t == "ident") {
        isRange = (peekTypeTrimmed(ts) == "period");
    }
    if (isRange) {
        node->addChild(parseRange(ts));
    } else if (ts.check("ident")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
    } else {
        throwSyntaxError(ts.current(), "range atau ident di dalam bracket
array", ts.getIndex());
    }
    if (!ts.check("rbrack")) {
        throwSyntaxError(ts.current(), "rbrack", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("ofsy")) {
        throwSyntaxError(ts.current(), "ofsy", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    node->addChild(parseType(ts));
    return node;
}

Node* parseFieldPart(TokenStream& ts) {

```

```

    Node* node = new Node("<field-part>");
    node->addChild(parseIdentifierList(ts));
    if (!ts.check("colon")) {
        throwSyntaxError(ts.current(), "colon di field-part",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    node->addChild(parseType(ts));
    return node;
}

Node* parseFieldList(TokenStream& ts) {
    Node* node = new Node("<field-list>");
    node->addChild(parseFieldPart(ts));
    while (ts.check("semicolon")) {
        if (peekTypeTrimmed(ts) == "endsy") {
            break;
        }
        if (peekTypeTrimmed(ts) != "ident") {
            break;
        }
        node->addChild(new Node(ts.current()));
        ts.advance();
        node->addChild(parseFieldPart(ts));
    }
    return node;
}

Node* parseRecordType(TokenStream& ts) {
    Node* node = new Node("<record-type>");
    if (!ts.check("recordsy")) {
        throwSyntaxError(ts.current(), "recordsy", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    node->addChild(parseFieldList(ts));
    if (!ts.check("endsy")) {
        throwSyntaxError(ts.current(), "endsy penutup record",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    return node;
}

Node* parseType(TokenStream& ts) {
    Node* node = new Node("<type>");
    std::string t = ts.currentType();
    if (t == "arraysy") {
        node->addChild(parseArrayType(ts));
        return node;
    }
    if (t == "lparent") {

```



```

        node->addChild(parseEnumerated(ts));
        return node;
    }
    if (t == "recordsy") {
        node->addChild(parseRecordType(ts));
        return node;
    }
    if (t == "plus" || t == "minus") {
        node->addChild(parseRange(ts));
        return node;
    }
    if (t == "intcon" || t == "realcon" || t == "charcon" || t ==
"string") {
        node->addChild(parseRange(ts));
        return node;
    }
    if (t == "ident") {
        if (peekTypeTrimmed(ts) == "period") {
            node->addChild(parseRange(ts));
        } else {
            node->addChild(new Node(ts.current()));
            ts.advance();
        }
        return node;
    }
    throwSyntaxError(ts.current(), "type (ident | array-type | range |
enumerated | record-type)", ts.getIndex());
    return nullptr;
}

Node* parseConstDeclaration(TokenStream& ts) {
    Node* node = new Node("<const-declaration>");
    if (!ts.check("constsy")) {
        throwSyntaxError(ts.current(), "constsy", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("ident")) {
        throwSyntaxError(ts.current(), "ident (nama konstanta)",
ts.getIndex());
    }
    while (ts.check("ident")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
        if (!ts.check("eq1")) {
            throwSyntaxError(ts.current(), "eq1 (==) setelah nama
konstanta", ts.getIndex());
        }
        node->addChild(new Node(ts.current()));
        ts.advance();
        node->addChild(parseConstant(ts));
        if (!ts.check("semicolon")) {
            throwSyntaxError(ts.current(), "semicolon setelah nilai
konstanta", ts.getIndex());
        }
    }
}

```

```

    }
    node->addChild(new Node(ts.current()));
    ts.advance();
}
return node;
}

Node* parseTypeDeclaration(TokenStream& ts) {
    Node* node = new Node("<type-declaration>");
    if (!ts.check("typesy")) {
        throwSyntaxError(ts.current(), "typesy", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("ident")) {
        throwSyntaxError(ts.current(), "ident (nama type)",
ts.getIndex());
    }
    while (ts.check("ident")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
        if (!ts.check("eql")) {
            throwSyntaxError(ts.current(), "eql (==) setelah nama type",
ts.getIndex());
        }
        node->addChild(new Node(ts.current()));
        ts.advance();
        node->addChild(parseType(ts));
        if (!ts.check("semicolon")) {
            throwSyntaxError(ts.current(), "semicolon setelah definisi
type", ts.getIndex());
        }
        node->addChild(new Node(ts.current()));
        ts.advance();
    }
    return node;
}

Node* parseVarDeclaration(TokenStream& ts) {
    Node* node = new Node("<var-declaration>");
    if (!ts.check("varsy")) {
        throwSyntaxError(ts.current(), "varsy", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("ident")) {
        throwSyntaxError(ts.current(), "ident (nama variabel)",
ts.getIndex());
    }
    while (ts.check("ident")) {
        node->addChild(parseIdentifierList(ts));
        if (!ts.check("colon")) {
            throwSyntaxError(ts.current(), "colon setelah
identifier-list", ts.getIndex());

```

```

    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    node->addChild(parseType(ts));
    if (!ts.check("semicolon")) {
        throwSyntaxError(ts.current(), "semicolon setelah deklarasi
variabel", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
}
return node;
}

Node* parseParameterGroup(TokenStream& ts) {
    Node* node = new Node("<parameter-group>");
    node->addChild(parseIdentifierList(ts));
    if (!ts.check("colon")) {
        throwSyntaxError(ts.current(), "colon di parameter-group",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (ts.check("arraysy")) {
        node->addChild(parseArrayType(ts));
    } else if (ts.check("ident")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
    } else {
        throwSyntaxError(ts.current(), "ident atau array-type sebagai tipe
parameter", ts.getIndex());
    }
    return node;
}

Node* parseFormalParameterList(TokenStream& ts) {
    Node* node = new Node("<formal-parameter-list>");
    if (!ts.check("lparent")) {
        throwSyntaxError(ts.current(), "lparent", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    node->addChild(parseParameterGroup(ts));
    while (ts.check("semicolon")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
        node->addChild(parseParameterGroup(ts));
    }
    if (!ts.check("rparent")) {
        throwSyntaxError(ts.current(), "rparent penutup
formal-parameter-list", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
}

```

```

        return node;
    }

static Node* parseProcedureDeclaration(TokenStream& ts) {
    Node* node = new Node("<procedure-declaration>");
    if (!ts.check("proceduresy")) {
        throwSyntaxError(ts.current(), "proceduresy", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("ident")) {
        throwSyntaxError(ts.current(), "ident (nama prosedur)",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (ts.check("lparent")) {
        node->addChild(parseFormalParameterList(ts));
    }
    if (!ts.check("semicolon")) {
        throwSyntaxError(ts.current(), "semicolon setelah header
prosedur", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    node->addChild(parseBlock(ts));
    if (!ts.check("semicolon")) {
        throwSyntaxError(ts.current(), "semicolon penutup prosedur",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    return node;
}

static Node* parseFunctionDeclaration(TokenStream& ts) {
    Node* node = new Node("<function-declaration>");
    if (!ts.check("functionsy")) {
        throwSyntaxError(ts.current(), "functionsy", ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("ident")) {
        throwSyntaxError(ts.current(), "ident (nama fungsi)",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (ts.check("lparent")) {
        node->addChild(parseFormalParameterList(ts));
    }
    if (!ts.check("colon")) {
        throwSyntaxError(ts.current(), "colon sebelum return type",
ts.getIndex());
    }

```

```

    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("ident")) {
        throwSyntaxError(ts.current(), "ident (return type)",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    if (!ts.check("semicolon")) {
        throwSyntaxError(ts.current(), "semicolon setelah header fungsi",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    node->addChild(parseBlock(ts));
    if (!ts.check("semicolon")) {
        throwSyntaxError(ts.current(), "semicolon penutup fungsi",
ts.getIndex());
    }
    node->addChild(new Node(ts.current()));
    ts.advance();
    return node;
}

Node* parseSubprogramDeclaration(TokenStream& ts) {
    Node* node = new Node("<subprogram-declaration>");
    if (ts.check("proceduresy")) {
        node->addChild(parseProcedureDeclaration(ts));
    } else if (ts.check("functionsy")) {
        node->addChild(parseFunctionDeclaration(ts));
    } else {
        throwSyntaxError(ts.current(), "proceduresy atau functionsy",
ts.getIndex());
    }
    return node;
}

Node* parseBlock(TokenStream& ts) {
    Node* node = new Node("block");
    Node* declPart = new Node("<declaration-part>");
    bool hasDecl = false;
    while (ts.check("constsy") || ts.check("typesy") || ts.check("varsy")
|| ts.check("proceduresy") || ts.check("functionsy")) {
        hasDecl = true;
        if (ts.check("constsy"))
declPart->addChild(parseConstDeclaration(ts));
        else if (ts.check("typesy"))
declPart->addChild(parseTypeDeclaration(ts));
        else if (ts.check("varsy"))
declPart->addChild(parseVarDeclaration(ts));
        else declPart->addChild(parseSubprogramDeclaration(ts));
    }
    if (hasDecl) {

```

```

        node->addChild(declPart);
    } else {
        delete declPart;
    }
    node->addChild(parseCompoundStatement(ts));
    return node;
}

```

Bagian *declaration parser* bertanggung jawab untuk memproses seluruh struktur deklarasi yang muncul sebelum *compound statement* dalam grammar bahasa Arion. Implementasi ini mencakup berbagai jenis deklarasi, yaitu *const-declaration*, *type-declaration*, *var-declaration*, serta *subprogram-declaration* (procedure dan function). Seluruh fungsi pada modul ini diimplementasikan menggunakan pendekatan *Recursive Descent Parser*, di mana setiap fungsi merepresentasikan satu aturan produksi dalam grammar.

Secara umum, proses parsing deklarasi diawali dari fungsi `parseDeclarationPartCoordinator` (di core parser), yang kemudian akan memanggil fungsi-fungsi spesifik seperti `parseConstDeclaration`, `parseTypeDeclaration`, `parseVarDeclaration`, dan `parseSubprogramDeclaration` berdasarkan token yang sedang dibaca. Pendekatan ini memungkinkan parser untuk secara fleksibel menangani urutan deklarasi yang berulang sesuai spesifikasi grammar.

Pada implementasi *const-declaration*, fungsi `parseConstDeclaration` akan membaca token `constsy`, diikuti dengan satu atau lebih pasangan *identifier* dan nilai konstanta. Nilai konstanta diproses melalui fungsi `parseConstant`, yang mendukung berbagai tipe seperti `charcon`, `string`, `intcon`, `realcon`, serta *identifier* dengan kemungkinan tanda unary (*plus* atau *minus*). Setiap

deklarasi konstanta harus diakhiri dengan semicolon, dan jika tidak sesuai, parser akan menghasilkan *syntax error* yang informatif.

Untuk *type-declaration*, fungsi `parseTypeDeclaration` digunakan untuk mendefinisikan tipe baru. Implementasi ini mendukung berbagai bentuk tipe, termasuk tipe sederhana (*ident*), *array-type*, *range*, *enumerated*, dan *record-type*. Fungsi `parseType` berperan penting sebagai dispatcher yang menentukan jenis tipe berdasarkan token saat ini. Misalnya, jika token berupa `arraysy`, maka parser akan memanggil `parseArrayType`, sedangkan jika token berupa tanda kurung buka `lparent`, maka akan diproses sebagai *enumerated type* melalui `parseEnumerated`.

Pada *array-type*, parser memproses struktur `arraysy [ range | ident ] of type`. Implementasi ini cukup kompleks karena harus membedakan apakah isi indeks berupa *range* atau identifier biasa. Untuk itu digunakan fungsi tambahan seperti `peekTypeTrimmed` guna melihat token berikutnya tanpa mengubah posisi parsing untuk memastikan interpretasi grammar yang benar, terutama pada kasus ambigu. Selain itu, *record-type* diimplementasikan melalui fungsi `parseRecordType`, yang memanfaatkan `parseFieldList` dan `parseFieldPart` untuk membaca daftar field dalam record. Setiap field terdiri dari *identifier-list*, diikuti colon, dan tipe data. Struktur ini mendukung deklarasi kompleks seperti struktur data bertipe record yang memiliki banyak atribut.

Untuk *var-declaration*, fungsi `parseVarDeclaration` digunakan untuk membaca deklarasi variabel dalam bentuk `identifier-list : type ;`. Fungsi ini memanfaatkan kembali `parseIdentifierList` dan `parseType` sehingga modular dan konsisten dengan bagian deklarasi lainnya.

Bagian *subprogram-declaration* mencakup deklarasi prosedur dan fungsi, yang diimplementasikan melalui fungsi `parseProcedureDeclaration` dan `parseFunctionDeclaration`. Kedua fungsi ini mendukung parameter opsional melalui `parseFormalParameterList`, serta blok program melalui `parseBlock`. Fungsi `parseBlock` sendiri merupakan kombinasi dari *declaration-part* dan *compound-statement*, sehingga memungkinkan adanya deklarasi lokal di dalam prosedur atau fungsi. Seluruh fungsi dalam modul ini dilengkapi dengan mekanisme *error handling* menggunakan `throwSyntaxError`, yang akan memberikan pesan kesalahan spesifik ketika token yang ditemukan tidak sesuai dengan yang diharapkan. Hal ini membantu dalam

proses debugging serta memastikan parser tidak melanjutkan parsing pada kondisi sintaks yang tidak valid.

d. Implementasi Statement dan Control Flow Parser

```
#include "statement_parser.hpp"
#include "parser/utils/error.hpp"
#include "parser/expression/expression_parser.hpp"

Node* parseCompoundStatement(TokenStream& ts) {
    Node* node = new Node("<compound-statement>");

    node->addChild(new Node(ts.expect("beginsy", "compound statement")));

    node->addChild(parseStatementList(ts));

    node->addChild(new Node(ts.expect("endsy", "compound statement")));

    return node;
}

Node* parseStatementList(TokenStream& ts) {
    Node* node = new Node("<statement-list>");

    node->addChild(parseStatement(ts));

    while (ts.check("semicolon")) {
        node->addChild(new Node(ts.current()));
        ts.advance();

        if (ts.check("endsy") || ts.check("untilsy")) break;

        node->addChild(parseStatement(ts));
    }

    return node;
}
```



```

Node* parseStatement(TokenStream& ts) {
    if (ts.check("ident")) {
        int saveIndex = ts.getIndex();
        try {
            return parseAssignmentStatement(ts);
        } catch (...) {
            ts.setIndex(saveIndex);
            return parseProcedureFunctionCall(ts);
        }
    }
    else if (ts.check("ifsy")) return parseIfStatement(ts);
    else if (ts.check("whilesy")) return parseWhileStatement(ts);
    else if (ts.check("repeatsy")) return parseRepeatStatement(ts);
    else if (ts.check("forsy")) return parseForStatement(ts);
    else if (ts.check("casesy")) return parseCaseStatement(ts);
    else if (ts.check("beginsy")) return parseCompoundStatement(ts);

    if (ts.isEOF() || ts.check("endsy") || ts.check("untilsy") ||
ts.check("elsesy")) {
        return new Node("<statement>");
    }

    Node* node = new Node("<unknown-statement>");
    node->addChild(new Node(ts.current()));
    ts.advance();
    return node;
}

Node* parseAssignmentStatement(TokenStream& ts) {
    Node* node = new Node("<assignment-statement>");

    // node->addChild(new Node(ts.expect("ident", "assignment")));
    node->addChild(parseVariable(ts));
    node->addChild(new Node(ts.expect("becomes", "assignment")));

    node->addChild(parseExpression(ts));

    return node;
}

```

```

Node* parseProcedureFunctionCall(TokenStream& ts){
    Node* node = new Node("<procedure/function-call>");

    node->addChild(new Node(ts.expect("ident", "procedure call")));

    if (ts.check("lparent")) {
        node->addChild(new Node(ts.current()));
        ts.advance();

        if (!ts.check("rparent")) {
            // node->addChild(parseExpressionList(ts));
            node->addChild(parseParameterList(ts));
        }

        node->addChild(new Node(ts.expect("rparent", "procedure
call"))));
    }

    while (ts.check("ident")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
    }

    return node;
}

Node* parseIfStatement(TokenStream& ts){
    Node* node = new Node("<if-statement>");

    node->addChild(new Node(ts.expect("ifsy")));
    node->addChild(parseExpression(ts));
    node->addChild(new Node(ts.expect("thensy")));

    node->addChild(parseStatement(ts));

    if (ts.check("elsesy")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
        node->addChild(parseStatement(ts));
    }
}

```

```

        return node;
    }

Node* parseWhileStatement(TokenStream& ts) {
    Node* node = new Node("<while-statement>");

    node->addChild(new Node(ts.expect("whilesy")));
    node->addChild(parseExpression(ts));
    node->addChild(new Node(ts.expect("dosy")));

    node->addChild(parseStatement(ts));

    return node;
}

Node* parseRepeatStatement(TokenStream& ts) {
    Node* node = new Node("<repeat-statement>");

    node->addChild(new Node(ts.expect("repeatsy")));

    node->addChild(parseStatementList(ts));

    node->addChild(new Node(ts.expect("untilsy")));
    node->addChild(parseExpression(ts));

    return node;
}

Node* parseForStatement(TokenStream& ts) {
    Node* node = new Node("<for-statement>");

    node->addChild(new Node(ts.expect("forsy")));
    node->addChild(new Node(ts.expect("ident")));
    node->addChild(new Node(ts.expect("becomes")));

    node->addChild(parseExpression(ts));

    if (ts.check("tosy") || ts.check("downtosy")) {
        node->addChild(new Node(ts.current()));
    }
}

```

```

        ts.advance();
    } else {
        throwSyntaxError(ts.current(), "tosy/downtosy",
ts.getIndex());
    }

    node->addChild(parseExpression(ts));

    node->addChild(new Node(ts.expect("dosy")));
    node->addChild(parseStatement(ts));

    return node;
}

Node* parseCaseStatement(TokenStream& ts){
    Node* node = new Node("<case-statement>");

    node->addChild(new Node(ts.expect("casesy")));
    node->addChild(parseExpression(ts));
    node->addChild(new Node(ts.expect("ofsy")));

    node->addChild(parseCaseList(ts));

    node->addChild(new Node(ts.expect("endsy")));

    return node;
}

Node* parseCaseList(TokenStream& ts){
    Node* node = new Node("<case-list>");

    node->addChild(parseCaseBranch(ts));

    while (ts.check("semicolon")) {
        node->addChild(new Node(ts.current()));
        ts.advance();

        if (ts.check("endsy")) break;

        node->addChild(parseCaseBranch(ts));
    }
}

```

```

    }

    return node;
}

Node* parseCaseBranch(TokenStream& ts){
    Node* node = new Node("<case-branch>");

    node->addChild(parseConstantList(ts));
    node->addChild(new Node(ts.expect("colon")));

    node->addChild(parseStatement(ts));

    return node;
}

Node* parseConstantList(TokenStream& ts){
    Node* node = new Node("<constant-list>");

    node->addChild(parseExpression(ts));

    while (ts.check("comma")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
        node->addChild(parseExpression(ts));
    }

    return node;
}

// Node* parseExpressionList(TokenStream& ts){
//     Node* node = new Node("<expression-list>");

//     node->addChild(parseExpression(ts));

//     while (ts.check("comma")) {
//         node->addChild(new Node(ts.current()));
//         ts.advance();
//         node->addChild(parseExpression(ts));
//     }

```

```
//      return node;
// }
```

Bagian *statement parser* bertanggung jawab untuk menangani seluruh struktur aksi atau instruksi dalam program, khususnya yang berada di dalam *compound statement*. Implementasi ini mencakup berbagai bentuk statement sesuai grammar Arion, seperti *assignment*, *procedure/function call*, *if*, *while*, *repeat*, *for*, *case*, serta *compound statement* itu sendiri. Seluruh parsing dilakukan menggunakan pendekatan *Recursive Descent*, di mana setiap fungsi merepresentasikan satu aturan produksi grammar.

Parsing untuk blok utama dimulai dari fungsi `parseCompoundStatement`, yang mengimplementasikan aturan $\langle \text{compound-statement} \rangle \rightarrow \text{beginsy} \langle \text{statement-list} \rangle \text{endsy}$. Fungsi ini memastikan bahwa setiap blok program diawali dengan token `beginsy`, diikuti daftar statement, dan diakhiri dengan `endsy`. Jika urutan token tidak sesuai, maka akan dihasilkan *syntax error* melalui fungsi `expect`. Selanjutnya, fungsi `parseStatementList` digunakan untuk memproses daftar statement yang dipisahkan oleh semicolon. Implementasi ini menggunakan loop untuk membaca statement secara berulang hingga tidak ditemukan semicolon lagi, atau ketika mencapai token penutup seperti `endsy` atau `untilsy`. Hal ini penting untuk memastikan bahwa parser tidak membaca melewati batas blok yang sedang diproses. Fungsi inti dari modul ini adalah `parseStatement`, yang bertugas menentukan jenis statement berdasarkan token saat ini. Pemilihan dilakukan menggunakan serangkaian pengecekan kondisi (*if-else*), seperti:

- Jika token adalah `ident`, maka akan dicek apakah merupakan *assignment* (jika diikuti `becomes`) atau *procedure/function call*.
- Jika token adalah `ifsy`, `whilesy`, `repeatsy`, `forsy`, atau `casesy`, maka akan dipanggil parser yang ~~hanya menangani~~.
- Jika token adalah `beginsy`, maka akan diproses sebagai *nested compound statement*.

Selain itu, fungsi ini juga menangani *empty statement* (ϵ), yaitu kondisi di mana tidak ada statement yang valid, misalnya sebelum `endsy`, `untilsy`, atau `elsesy`. Dalam kasus tersebut, parser akan tetap menghasilkan node kosong $\langle \text{statement} \rangle$ agar struktur parse tree tetap konsisten.

Untuk *assignment statement*, fungsi `parseAssignmentStatement` mengimplementasikan aturan `<assignment-statement> → variable becomes expression`. Berbeda dari implementasi sederhana yang hanya menerima identifier, parser ini menggunakan fungsi `parseVariable` sehingga mendukung bentuk variabel yang lebih kompleks seperti array indexing atau field access.

Pada *procedure/function call*, fungsi `parseProcedureFunctionCall` menangani pemanggilan dengan atau tanpa parameter. Jika ditemukan `lparent`, parser akan memproses daftar parameter melalui `parseParameterList`. Implementasi ini juga fleksibel untuk menangani variasi pemanggilan fungsi sesuai grammar.

Untuk struktur kontrol:

- If Statement (`parseIfStatement`) mengimplementasikan kondisi dengan opsi else, sesuai aturan `<if-statement> → ifsy expression thensy statement (elsesy statement)?`.
- While Statement (`parseWhileStatement`) memproses perulangan dengan kondisi di awal.
- Repeat Statement (`parseRepeatStatement`) memproses perulangan dengan kondisi di akhir, menggunakan `untilsy`.
- For Statement (`parseForStatement`) menangani perulangan dengan counter, termasuk arah iterasi `tosy` atau `downtosy`.
- Case Statement (`parseCaseStatement`) memproses percabangan multi-kondisi dengan bantuan fungsi `parseCaseList` dan `parseCaseBranch`.

Pada *case statement*, parser membaca daftar cabang kondisi melalui `parseCaseList`, di mana setiap cabang terdiri dari daftar konstanta (`parseConstantList`), diikuti colon, dan sebuah statement. Struktur ini memungkinkan penanganan banyak kondisi dalam satu blok case. Seluruh fungsi dalam modul ini terintegrasi dengan *expression parser*, khususnya untuk memproses kondisi dan nilai pada assignment maupun control flow. Selain itu, mekanisme *error handling* diterapkan secara ketat menggunakan fungsi `expect` dan `throwSyntaxError` sehingga setiap kesalahan sintaks dapat dideteksi secara spesifik dan informatif. Parser tidak hanya berhenti saat terjadi kesalahan, tetapi juga memberikan konteks yang jelas mengenai token yang tidak sesuai.

e. Implementasi Expression Parser

```
#include "expression_parser.hpp"
```

```

#include "parser/utils/error.hpp"

Node* parseExpression(TokenStream& ts) {
    Node* node = new Node("<expression>");

    node->addChild(parseSimpleExpression(ts));

    while (ts.check("eq1") || ts.check("neq") || ts.check("gtr") ||
ts.check("geq") || ts.check("lss") || ts.check("leq")) {
        node->addChild(new Node (ts.current()));
        ts.advance();

        node->addChild(parseSimpleExpression(ts));
    }

    return node;
}

Node* parseSimpleExpression(TokenStream& ts) {
    Node* node = new Node("<simple-expression>");

    while (ts.check("plus") || ts.check("minus") || ts.check("orsy")) {
        node->addChild(new Node (ts.current()));
        ts.advance();
    }

    node->addChild(parseTerm(ts));

    while (ts.check("plus") || ts.check("minus") || ts.check("orsy")) {
        // node->addChild(new Node(ts.current())); // operator
        node->addChild(parseAdditiveOperator(ts));
        // ts.advance();

        node->addChild(parseTerm(ts));
    }

    return node;
}

```



```

Node* parseTerm(TokenStream& ts){
    Node* node = new Node("<term>");

    node->addChild(parseFactor(ts));

    while (ts.check("times") || ts.check("rdiv") || ts.check("idiv") ||
ts.check("imod") || ts.check("andsy")) {
        node->addChild(new Node(ts.current())); // operator
        ts.advance();

        node->addChild(parseFactor(ts));
    }

    return node;
}

Node* parseVariableTanpaIdent (TokenStream& ts) {
    Node* node = new Node ("<variable>");

    while (ts.check("lbrack") || ts.check("period")) {
        // std::cout << "masuk loop" << std::endl;
        node->addChild(parseComponentVariable(ts));
    }

    // node->addChild(parseComponentVariable(ts));

    return node;
}

Node* parseProcedureCallTanpaIdent (TokenStream& ts) {
    Node* node = new Node("<procedure-call>");

    if (ts.check("lparent")) {
        node->addChild(new Node(ts.current()));
        ts.advance();

        if (!ts.check("rparent")) {
            node->addChild(parseParameterList(ts));
        }
    }
}

```

```

        node->addChild(new Node(ts.expect("rparent", "procedure call")));
    }

    while (ts.check("ident")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
    }

    return node;
}

Node* parseFactor(TokenStream& ts) {
    Node* node = new Node("<factor>");

    if (ts.check("intcon") || ts.check("realcon") || ts.check("charcon")
|| ts.check("string")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
    }
    else if (ts.check("ident")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
    }

    if (ts.check("lbrack") || ts.check("period")) { // variabel
        node->addChild(parseVariableTanpaIdent(ts));
    }
    else if (ts.check("lparent")) { // procedure call
        node->addChild(parseProcedureCallTanpaIdent(ts));
    }

}
else if (ts.check("lparent")) {
    node->addChild(new Node(ts.current())); // (
    ts.advance();

    node->addChild(parseExpression(ts));

    node->addChild(new Node(ts.expect("rparent", "factor")));
}
else if (ts.check("notsy")) {

```

```

        node->addChild(new Node(ts.current()));
        ts.advance();
        node->addChild(parseFactor(ts));
    }
    else {
        throwSyntaxError(ts.current(), "factor", ts.getIndex());
    }

    return node;
}

```

```

Node* parseVariable (TokenStream& ts) {
    if (ts.getIndex() > 90) {
        std::cout << "masuk vairabel\n";
        std::cout << ts.current() << std::endl;
    }
    Node* node = new Node("<variable>");

    if (ts.check("ident")) {
        node->addChild(new Node(ts.current()));
        ts.advance();
    }
    // else {
    std::cout << "masuk sini" << std::endl;
    while (ts.check("lbrack") || ts.check("period")) {
        std::cout << "masuk loop" << std::endl;
        node->addChild(parseComponentVariable(ts));
    }
    // }

    return node;
}

```

```

Node* parseComponentVariable (TokenStream& ts) {
    Node* node = new Node("<component-variable>");

    // node->addChild(parseVariable(ts));

    if (ts.check("lbrack")) {
        node->addChild (new Node (ts.current()));
    }
}

```

```

        ts.advance();

        node->addChild (parseIndexList(ts));

        if (ts.check("rbrack")) {
            node->addChild (new Node (ts.current()));
            ts.advance();
        }
        else {
            throwSyntaxError(ts.current(), "rbrack", ts.getIndex());
        }
    }
    else if (ts.check("period")) {
        node->addChild (new Node (ts.current()));
        ts.advance();

        if (ts.check("ident")) {
            node->addChild (new Node (ts.current()));
            ts.advance();
        }
        else {
            throwSyntaxError(ts.current(), "ident", ts.getIndex());
        }
    }
    else {
        throwSyntaxError(ts.current(), "[ or .", ts.getIndex());
    }

    return node;
}

Node* parseIndexList (TokenStream& ts) {
    Node* node = new Node ("<index-list>");

    if (ts.check("intcon") || ts.check("charcon") || ts.check("ident")) {
        node->addChild (new Node(ts.current()));
        ts.advance();

        while (ts.check("comma")) {
            node->addChild (new Node(ts.current()));

```

```

        ts.advance();

        node->addChild (parseIndexList(ts));
    }
}
else {
    throwSyntaxError(ts.current(), "intcon, charcon, ident",
ts.getIndex());
}

return node;
}

// Node* parseAssignmentStatement (TokenStream& ts) {
//     Node* node = new Node ("<assignment-statement>");

//     node->addChild(parseVariable(ts));

//     if (ts.check("becomes")) {
//         node->addChild(new Node (ts.current()));
//         ts.advance();

//         node->addChild(parseExpression(ts));
//     }
//     else {
//         throwSyntaxError (ts.current(), "becomes", ts.getIndex());
//     }

//     return node;
// }

Node* parseParameterList (TokenStream& ts) {
    Node* node = new Node ("<parameter-list>");

    node->addChild(parseExpression(ts));

    while (ts.check("comma")) {
        node->addChild(new Node (ts.current()));
        ts.advance();
    }
}

```

```

        node->addChild(parseExpression(ts));
    }

    return node;
}

Node* parseRelationalOperator (TokenStream& ts) {
    Node* node = new Node("<relational-operator>");
    // eql | neq | gtr | geq | lss | leq
    if (ts.check("eql") || ts.check("neq") || ts.check("gtr") ||
ts.check("geq") || ts.check("lss") || ts.check("leq")) {
        node->addChild (new Node(ts.current()));
        ts.advance();
    }
    else {
        throwSyntaxError(ts.current(), "relational operator",
ts.getIndex());
    }

    return node;
}

Node* parseAdditiveOperator (TokenStream& ts) {
    Node* node = new Node("<additive-operator>");
    // eql | neq | gtr | geq | lss | leq
    if (ts.check("plus") || ts.check("minus") || ts.check("orsy")) {
        node->addChild (new Node(ts.current()));
        ts.advance();
    }
    else {
        throwSyntaxError(ts.current(), "additive operator",
ts.getIndex());
    }

    return node;
}

Node* parseMultiplicativeOperator (TokenStream& ts) {
    Node* node = new Node("<multiplicative-operator>");
    // eql | neq | gtr | geq | lss | leq

```

```

        if (ts.check("times") || ts.check("rdiv") || ts.check("idiv") ||
ts.check("imod") || ts.check("andsy")) {
            node->addChild (new Node(ts.current()));
            ts.advance();
        }
        else {
            throwSyntaxError(ts.current(), "multiplicative operator",
ts.getIndex());
        }

        return node;
    }
}

```

Bagian *expression parser* bertanggung jawab untuk memproses seluruh bentuk ekspresi dalam grammar bahasa Arion. Implementasi ini mencakup parsing untuk *expression*, *simple-expression*, *term*, *factor*, operator aritmatika maupun relasional, parameter pemanggilan fungsi, serta variable kompleks seperti array dan record field. Modul ini menjadi salah satu komponen penting karena digunakan oleh berbagai bagian parser lain, terutama pada assignment, kondisi percabangan, dan perulangan.

Implementasi dilakukan menggunakan pendekatan *Recursive Descent Parser*, di mana setiap aturan grammar direpresentasikan dalam satu fungsi khusus. Struktur parser disusun secara bertingkat untuk menjaga prioritas operator sesuai aturan evaluasi ekspresi. Fungsi utama pada modul ini adalah `parseExpression`, yang mengimplementasikan aturan:

$$\langle \text{expression} \rangle \rightarrow \langle \text{simple-expression} \rangle$$

$$(\text{relational-operator } \langle \text{simple-expression} \rangle)?$$

Fungsi ini terlebih dahulu memproses *simple-expression*, kemudian memeriksa apakah terdapat operator relasional seperti `eq`, `neq`, `gtr`, `geq`, `lss`, atau `leq`. Jika ditemukan operator relasional, parser akan menambahkan operator tersebut ke parse tree dan melanjutkan parsing terhadap *simple-expression* berikutnya.

Pada level berikutnya, fungsi `parseSimpleExpression` menangani operator aditif seperti plus, minus, dan orsy. Fungsi ini juga mendukung unary operator di awal ekspresi. Setelah operator unary diproses, parser akan memanggil `parseTerm` untuk menangani operand utama. Selanjutnya, parser akan terus membaca operator aditif selama token yang ditemukan masih sesuai grammar.

Struktur prioritas operator dilanjutkan pada fungsi `parseTerm` yang menangani operator dengan prioritas lebih tinggi seperti:

- times
- rdiv
- idiv
- imod
- andsy

Fungsi ini memanggil `parseFactor` sebagai unit terkecil ekspresi. Dengan pemisahan antara *expression*, *simple-expression*, dan *term*, parser dapat mempertahankan urutan prioritas operator secara benar sesuai grammar bahasa Arion. Bagian paling penting dalam modul ini adalah fungsi `parseFactor` karena fungsi ini menangani berbagai kemungkinan bentuk operand dalam ekspresi. Parser mendukung:

- integer constant (`intcon`)
- real constant (`realcon`)
- character constant (`charcon`)
- string constant (`string`)
- variable
- procedure/function call
- nested expression menggunakan tanda kurung
- operator not

Jika token berupa identifier (`ident`), parser akan melakukan pengecekan lanjutan untuk menentukan apakah identifier tersebut merupakan variable biasa, component variable, atau pemanggilan procedure/function. Untuk mendukung variable kompleks, implementasi

menggunakan fungsi `parseVariable` dan `parseComponentVariable`. Fungsi ini memungkinkan parser membaca bentuk seperti:

`arrayVar[1]`

`recordVar.field`

`matrix[1][2]`

Pada *component variable*, parser mendukung dua bentuk utama:

1. Indexing menggunakan lbrack ... rbrack
2. Field access menggunakan period ident

Sementara itu, daftar indeks array diproses oleh fungsi `parseIndexList` yang mendukung banyak indeks yang dipisahkan dengan comma. Selain variable, parser juga mendukung *procedure/function call* di dalam ekspresi melalui fungsi `parseProcedureCallTanpaIdent`. Implementasi ini memungkinkan pemanggilan fungsi seperti `sum(a, b)` dan `max(x, y)`

Parameter aktual diproses menggunakan fungsi `parseParameterList` yang membaca daftar ekspresi yang dipisahkan koma. Untuk meningkatkan modularitas dan keterbacaan kode, operator relasional, aditif, dan multiplicative juga dipisahkan ke dalam fungsi:

- `parseRelationalOperator`
- `parseAdditiveOperator`
- `parseMultiplicativeOperator`

Meskipun beberapa operator telah langsung ditangani dalam loop parsing utama, fungsi-fungsi ini tetap disediakan agar struktur parser lebih konsisten dengan grammar dan mudah dikembangkan pada milestone berikutnya. Seluruh fungsi pada modul ini telah dilengkapi dengan mekanisme *error handling* menggunakan `throwSyntaxError`. Jika token yang ditemukan tidak sesuai dengan grammar yang diharapkan, parser akan menghasilkan pesan error yang spesifik dan informatif sehingga mempermudah proses debugging.

f. Implementasi Parse Tree

```

#include "node.hpp"

// Constructor untuk membuat node dengan label
Node::Node(const std::string& label) {
    this->label = label;
}

// Tambahkan child ke node ini
void Node::addChild(Node* child) {
    this->children.push_back(child);
}

std::string Node::getLabel () {
    return this->label;
}

// Destructor untuk cleanup memory
Node::~~Node() {
    for(Node* child : children){
        delete child;
    }
}

```

Pada milestone ini, hasil utama dari proses parsing adalah sebuah *Parse Tree* yang merepresentasikan struktur sintaks program berdasarkan grammar bahasa Arion. Untuk membangun struktur pohon tersebut, digunakan kelas Node sebagai representasi dasar setiap simpul (*node*) dalam parse tree.

Implementasi parse tree dilakukan pada file node.cpp dan node.hpp. Setiap node merepresentasikan satu elemen grammar, baik berupa *non-terminal* seperti <expression> dan <statement>, maupun token terminal seperti ident, intcon, atau semicolon. Struktur utama parse tree diimplementasikan menggunakan pendekatan pohon (*tree structure*) dengan hubungan parent-child. Setiap node memiliki:

- sebuah label (label) untuk menyimpan nama grammar atau token,
- serta daftar anak (children) yang menyimpan node-node turunannya.

Constructor pada kelas Node digunakan untuk membuat node baru dengan label tertentu.

Saat parser menemukan bagian grammar tertentu, parser akan membuat node baru menggunakan constructor ini. Sebagai contoh, ketika parser membaca aturan `<expression>`, maka parser akan membuat node dengan label "`<expression>`". Untuk membangun hubungan hierarkis antar node, digunakan fungsi `addChild`. Fungsi ini digunakan di seluruh parser untuk menambahkan subtree hasil parsing ke node induknya. Sebagai contoh, pada parsing assignment statement, node `<assignment-statement>` akan memiliki child berupa `<variable>`, token `becomes`, dan `<expression>`. Dengan pendekatan ini, struktur parse tree dapat dibangun secara bertahap mengikuti proses recursive descent parser. Setiap fungsi parser akan:

1. membuat node untuk grammar saat ini,
2. memanggil parser grammar turunannya,
3. lalu menambahkan hasil parsing tersebut sebagai child.

Selain itu, kelas Node juga menyediakan fungsi `getLabel` untuk mengambil label dari node. Fungsi ini digunakan terutama pada proses pencetakan parse tree agar isi node dapat ditampilkan dengan format yang sesuai.

Karena parse tree dibangun secara dinamis menggunakan alokasi memori (`new Node(...)`), diperlukan mekanisme pembersihan memori untuk mencegah *memory leak*. Oleh karena itu, diimplementasikan destructor pada kelas Node. Destructur ini akan menghapus seluruh child secara rekursif ketika sebuah node dihapus. Dengan demikian, ketika root parse tree dihapus pada akhir program, seluruh struktur pohon akan otomatis dibersihkan dari memori.

Implementasi parse tree ini dirancang sederhana namun fleksibel sehingga dapat digunakan oleh seluruh modul parser, termasuk declaration parser, statement parser, dan expression parser. Struktur berbasis node juga mempermudah proses traversal, pencetakan tree, serta pengembangan lebih lanjut pada milestone berikutnya seperti semantic analysis atau Abstract Syntax Tree (AST).

g. Implementasi Token Stream dan Utility

Modul token stream

```

#include "token_stream.hpp"
#include "error.hpp"

// Inisialisasi dengan token list dari lexer
TokenStream::TokenStream(const std::vector<std::string>& tokenList) {
    this->tokens = tokenList;
    if(tokenList.size() > 0){
        this->currentIndex = 0;
    }
    else{
        this->currentIndex = -1;
    }
}

// Kembalikan token saat ini (tanpa maju)
std::string TokenStream::current() const{
    return this->tokens.at(this->currentIndex);
}

// Kembalikan tipe token saat ini (tanpa value)
// Contoh: "ident(Hello)" → "ident"
std::string TokenStream::currentType() const{
    std::string curr = this->current();
    std::string type = "";

    int i = 0;
    while (i < (int)curr.size() && curr[i] != '('){
        type.push_back(curr[i]);
        i++;
    }

    while (!type.empty() && type.back() == ' '){
        type.pop_back();
    }

    return type;
}

// Kembalikan value token saat ini
// Contoh: "ident(Hello)" → "Hello"

```

```

std::string TokenStream::currentValue() const{
    std::string curr = this->current();
    int i = 0;
    while(i < (int)curr.size() && curr[i] != '('){
        i++;
    }

    if(i >= (int)curr.size()) return "";

    std::string value = "";
    int j = i + 1;
    while(j < (int)curr.size() && curr[j] != ')'){
        value.push_back(curr[j]);
        j++;
    }

    return value;
}

// Maju ke token berikutnya (seperti ADV() di lexer)
void TokenStream::advance() {
    if(this->currentIndex + 1 < (int)this->tokens.size()){
        this->currentIndex++;
    }
}

// Cek apakah sudah habis
bool TokenStream::isEOF() const{
    return this->currentIndex >= (int)tokens.size();
}

// Cek apakah token saat ini bertipe tertentu
bool TokenStream::check(const std::string& type) const{
    return this->currentType() == type;
}

// Peek token berikutnya tanpa maju
std::string TokenStream::peek() const{
    if(this->currentIndex + 1 < (int)this->tokens.size()) {
        return this->tokens.at(this->currentIndex + 1);
    }
}

```

```

    }

    return "";
}

std::string TokenStream::peekType() const{
    std::string curr = this->peek();
    std::string type = "";

    int i = 0;
    while (i < (int)curr.size() && curr[i] != '('){
        type.push_back(curr[i]);
        i++;
    }

    return type;
}

// Posisi saat ini (berguna untuk error message)
int TokenStream::getIndex() const{
    return this->currentIndex;
}

void TokenStream::setIndex(int index){
    if (index >= 0 && index < (int)this->tokens.size()) {
        this->currentIndex = index;
    }
}

std::string TokenStream::expect(const std::string& type, const
std::string& context){
    if (!check(type)) {
        // lempar error - akan diimplementasikan setelah error.hpp selesai
        // throw std::runtime_error(
        //     "Syntax error: unexpected '" + current() + "', expected '"
+ type + "'" + (context.empty() ? "" : " in " + context));
        throwSyntaxError(current(), type, currentIndex);
    }

    std::string tok = current();
    advance();
}

```

```

    return tok;
}

```

Modul error handler

```

#include "error.hpp"
#include <sstream>

SyntaxError::SyntaxError(const std::string& message)
    : std::runtime_error(message) {}

void throwSyntaxError(const std::string& unexpected, const std::string&
expected, int tokenIndex) {
    std::ostringstream oss;
    oss << "Syntax error at token " << tokenIndex << ": unexpected '" <<
unexpected << "', expected '" << expected << "'";
    throw SyntaxError(oss.str());
}

void throwSyntaxError(const std::string& message, int tokenIndex) {
    std::ostringstream oss;
    oss << "Syntax error at token " << tokenIndex << ": " << message;
    throw SyntaxError(oss.str());
}

```

Pada implementasi parser, digunakan sebuah komponen utilitas bernama TokenStream yang berfungsi sebagai penghubung antara lexer dan parser. Komponen ini bertugas mengelola aliran token hasil lexical analysis agar parser dapat membaca token satu per satu secara terstruktur. Dengan adanya TokenStream, setiap parser tidak perlu mengakses vector token secara langsung sehingga proses parsing menjadi lebih modular, rapi, dan mudah dikembangkan.

Class TokenStream diinisialisasi menggunakan list token hasil lexer yang disimpan dalam `std::vector<std::string>`. Selain menyimpan token, class ini juga menyimpan indeks token saat ini (`currentIndex`) untuk melacak posisi parser ketika melakukan pembacaan token. Pada

constructor, indeks akan diatur ke posisi awal token apabila token tersedia, sedangkan apabila token kosong maka indeks diatur menjadi -1.

Fungsi utama yang digunakan parser adalah `current()`, yaitu untuk mengambil token saat ini tanpa memindahkan posisi pembacaan token. Selain itu, terdapat fungsi `currentType()` yang digunakan untuk mengambil tipe token tanpa nilai tokennya. Sebagai contoh, token `ident(Hello)` akan dikembalikan menjadi `ident`. Fungsi ini sangat penting karena parser melakukan pengecekan grammar berdasarkan tipe token, bukan isi token secara langsung. Selain mengambil tipe token, sistem juga menyediakan fungsi `currentValue()` untuk mengambil isi token di dalam tanda kurung. Sebagai contoh, token `ident(Hello)` akan menghasilkan nilai `Hello`. Fungsi ini berguna apabila parser atau tahap lanjutan compiler membutuhkan nilai literal dari token.

Untuk berpindah ke token berikutnya digunakan fungsi `advance()`. Fungsi ini bekerja seperti mekanisme `ADV()` pada lexer milestone sebelumnya. Parser akan memanggil fungsi ini setiap kali suatu token berhasil dikenali sesuai grammar. Dalam proses recursive descent parser, sering kali diperlukan pengecekan apakah token saat ini sesuai dengan token yang diharapkan. Oleh karena itu, dibuat fungsi `check()` yang membandingkan tipe token saat ini dengan tipe token tertentu. Selain itu, digunakan juga fungsi `peek()` dan `peekType()` untuk melihat token berikutnya tanpa memajukan posisi parser. Fitur ini digunakan pada beberapa grammar yang membutuhkan *lookahead*, misalnya untuk membedakan assignment statement dengan procedure/function call.

Untuk mempermudah validasi token, dibuat fungsi `expect()`. Fungsi ini akan memeriksa apakah token saat ini sesuai dengan token yang diharapkan. Jika sesuai, token akan dikembalikan dan parser berpindah ke token berikutnya menggunakan `advance()`. Namun apabila token tidak sesuai, sistem akan langsung melempar *syntax error*. Selain `TokenStream`, sistem parser juga dilengkapi dengan utility error handling pada file `error.cpp` dan `error.hpp`. Utility ini digunakan untuk menghasilkan pesan kesalahan sintaks yang informatif sehingga mempermudah proses debugging ketika input program tidak sesuai grammar.

Class `SyntaxError` dibuat sebagai turunan dari `std::runtime_error` sehingga error parser dapat ditangani menggunakan mekanisme exception pada C++. Kemudian dibuat fungsi `throwSyntaxError()` yang menghasilkan pesan error lengkap beserta posisi token saat kesalahan

terjadi. Pesan error mencakup token yang ditemukan serta token yang seharusnya muncul sesuai grammar.

Pengujian

1. Test Case 1

Pengujian kasus sederhana secara keseluruhan sesuai pada file input sesuai contoh pada dokumen spesifikasi.

input-1.txt
<pre>program Hello; var a, b: integer; begin a := 5; b := a + 10; writeln('Result = ', b); end.</pre>
output-1-lexer.txt
<pre>programsy ident (Hello) semicolon varsy ident (a) comma ident (b) colon ident (integer) semicolon beginsy ident (a) becomes intcon (5) semicolon ident (b) becomes ident (a)</pre>

```

plus
intcon (10)
semicolon
ident (writeln)
lparent
string (Result = )
comma
ident (b)
rparent
semicolon
endsy
period

```

output-1-parser.txt

```

<program>
  <program-header>
    programsy
    ident (Hello)
    semicolon
  <declaration-part>
    <var-declaration>
      varsy
      <identifier-list>
        ident (a)
        comma
        ident (b)
      colon
      <type>
        ident (integer)
      semicolon
    <compound-statement>
      beginsy
      <statement-list>
        <assignment-statement>
          <variable>
            ident (a)
          becomes
          <expression>
            <simple-expression>
              <term>
                <factor>
                  intcon (5)
          semicolon
        <assignment-statement>
          <variable>
            ident (b)

```

```

becomes
  <expression>
    <simple-expression>
      <term>
        <factor>
          ident (a)
        <additive-operator>
          plus
      <term>
        <factor>
          intcon (10)
    semicolon
  <procedure/function-call>
    ident (writeln)
    lparent
    <parameter-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
              string (Result = )
          comma
          <expression>
            <simple-expression>
              <term>
                <factor>
                  ident (b)
            rparent
          semicolon
    endsy
  period

```

2. Test Case 2

Pengujian kasus tidak error pada percabangan menggunakan *if*

input-2.txt

```

program OpsTest;

var
  x, y: integer;

```

```

begin
  x := 10;
  y := 5;

  if x == y then
    x := x + y;
  if x <> y then
    x := x - y;
  if x > y then
    x := x * y;
  if x >= y then
    x := x div y;
  if x < y then
    x := x / y;
  if x <= y then
    x := x MOD y;

  if NOT (x == y) AND (x > y) OR (y < x) then
    x := y;
end.

```

output-2-lexer.txt

```

programsy
ident (OpsTest)
semicolon
varsy
ident (x)
comma
ident (y)
colon
ident (integer)
semicolon
beginsy
ident (x)
becomes
intcon (10)
semicolon
ident (y)
becomes
intcon (5)
semicolon
ifsy
ident (x)
eq1
ident (y)

```

```
thensy
ident (x)
becomes
ident (x)
plus
ident (y)
semicolon
ifsy
ident (x)
neq
ident (y)
thensy
ident (x)
becomes
ident (x)
minus
ident (y)
semicolon
ifsy
ident (x)
gtr
ident (y)
thensy
ident (x)
becomes
ident (x)
times
ident (y)
semicolon
ifsy
ident (x)
geq
ident (y)
thensy
ident (x)
becomes
ident (x)
idiv
ident (y)
semicolon
ifsy
ident (x)
lss
ident (y)
thensy
ident (x)
```

```
becomes
ident (x)
rdiv
ident (y)
semicolon
ifsy
ident (x)
leq
ident (y)
thensy
ident (x)
becomes
ident (x)
imod
ident (y)
semicolon
ifsy
notsy
lparent
ident (x)
eq1
ident (y)
rparent
andsy
lparent
ident (x)
gtr
ident (y)
rparent
orsy
lparent
ident (y)
lss
ident (x)
rparent
thensy
ident (x)
becomes
ident (y)
semicolon
endsy
period
```

output-2-parser.txt

```
<program>
  <program-header>
```

```

    programsy
    ident (OpsTest)
    semicolon
<declaration-part>
    <var-declaration>
        varsy
        <identifier-list>
            ident (x)
            comma
            ident (y)
        colon
        <type>
            ident (integer)
        semicolon
<compound-statement>
    beginsy
    <statement-list>
        <assignment-statement>
            <variable>
                ident (x)
            becomes
            <expression>
                <simple-expression>
                    <term>
                        <factor>
                            intcon (10)
            semicolon
        <assignment-statement>
            <variable>
                ident (y)
            becomes
            <expression>
                <simple-expression>
                    <term>
                        <factor>
                            intcon (5)
            semicolon
        <if-statement>
            ifsy
            <expression>
                <simple-expression>
                    <term>
                        <factor>
                            ident (x)
            eql
            <simple-expression>

```

```

        <term>
            <factor>
                ident (y)
    thensy
    <assignment-statement>
        <variable>
            ident (x)
        becomes
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        ident (x)
                <additive-operator>
                    plus
                <term>
                    <factor>
                        ident (y)
    semicolon
    <if-statement>
        ifsy
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        ident (x)
        neq
        <simple-expression>
            <term>
                <factor>
                    ident (y)
    thensy
    <assignment-statement>
        <variable>
            ident (x)
        becomes
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        ident (x)
                <additive-operator>
                    minus
                <term>
                    <factor>
                        ident (y)

```



```

semicolon
<if-statement>
  ifsy
  <expression>
    <simple-expression>
      <term>
        <factor>
          ident (x)

    gtr
    <simple-expression>
      <term>
        <factor>
          ident (y)

  thensy
  <assignment-statement>
    <variable>
      ident (x)
    becomes
    <expression>
      <simple-expression>
        <term>
          <factor>
            ident (x)
          times
          <factor>
            ident (y)

semicolon
<if-statement>
  ifsy
  <expression>
    <simple-expression>
      <term>
        <factor>
          ident (x)

    geq
    <simple-expression>
      <term>
        <factor>
          ident (y)

  thensy
  <assignment-statement>
    <variable>
      ident (x)
    becomes
    <expression>
      <simple-expression>

```

```

                                <term>
                                <factor>
                                    ident (x)
                                idiv
                                <factor>
                                    ident (y)
semicolon
<if-statement>
    ifsy
    <expression>
        <simple-expression>
            <term>
                <factor>
                    ident (x)
            lss
            <simple-expression>
                <term>
                    <factor>
                        ident (y)
    thensy
    <assignment-statement>
        <variable>
            ident (x)
        becomes
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        ident (x)
                rdiv
                <factor>
                    ident (y)
semicolon
<if-statement>
    ifsy
    <expression>
        <simple-expression>
            <term>
                <factor>
                    ident (x)
            leq
            <simple-expression>
                <term>
                    <factor>
                        ident (y)
    thensy

```

```

    <assignment-statement>
      <variable>
        ident (x)
      becomes
      <expression>
        <simple-expression>
          <term>
            <factor>
              ident (x)
            imod
            <factor>
              ident (y)
semicolon
<if-statement>
  ifsy
  <expression>
    <simple-expression>
      <term>
        <factor>
          notsy
          <factor>
            lparent
            <expression>
              <simple-expression>
                <term>
                  <factor>
                    ident (x)
              eql
              <simple-expression>
                <term>
                  <factor>
                    ident (y)
            rparent
          andsy
          <factor>
            lparent
            <expression>
              <simple-expression>
                <term>
                  <factor>
                    ident (x)
              gtr
              <simple-expression>
                <term>
                  <factor>
                    ident (y)

```

```

                                rparent
                                <additive-operator>
                                orsy
                                <term>
                                <factor>
                                lparent
                                <expression>
                                <simple-expression>
                                <term>
                                <factor>
                                ident (y)
                                lss
                                <simple-expression>
                                <term>
                                <factor>
                                ident (x)
                                rparent
                                thensy
                                <assignment-statement>
                                <variable>
                                ident (x)
                                becomes
                                <expression>
                                <simple-expression>
                                <term>
                                <factor>
                                ident (y)
                                semicolon
                                endsy
                                period

```

3. Test Case 3

Pengujian untuk kasus dengan banyak error

input-3.txt
output-3-lexer.txt
output-3-parser.txt

--

4. Test Case 4

Pengujian pada kasus program yang sangat pendek, tetapi tetap valid.

input-4.txt
<pre>program tesprogramkecil; begin end.</pre>
output-4-lexer.txt
<pre>programsy ident (tesprogramkecil) semicolon beginsy endsy period</pre>
output-4-parser.txt
<pre><program> <program-header> programsy ident (tesprogramkecil) semicolon <declaration-part> <compound-statement> beginsy <statement-list> <statement> endsy period</pre>

5. Test Case 5

Pengujian kasus penggunaan komentar yang valid serta penggunaan *looping*

input-5.txt

```
program CommentTest;

var
  x: integer;
  i: integer;

begin
  { ini komentar biasa }
  x := 5;

  (* ini komentar
    multi-line *)

  x := x + 10;

  { komentar dengan simbol aneh + - * / := }

  x := 'Not a { comment }';

  for i := 1 to 10 do
    i := i + 1;

  for i := 10 downto 1 do
    i := i - 1;

  while i > 0 do
    i := i - 1;

  repeat
    i := i + 1;
  until i == 10;

  case i of
    1: i := 100;
    2: i := 200;
  end;
end.
```

output-5-lexer.txt

```
programsy
ident (CommentTest)
semicolon
varsy
ident (x)
```

```
colon
ident (integer)
semicolon
ident (i)
colon
ident (integer)
semicolon
beginsy
ident (x)
becomes
intcon (5)
semicolon
ident (x)
becomes
ident (x)
plus
intcon (10)
semicolon
ident (x)
becomes
string (Not a { comment })
semicolon
forsy
ident (i)
becomes
intcon (1)
tosy
intcon (10)
dosy
ident (i)
becomes
ident (i)
plus
intcon (1)
semicolon
forsy
ident (i)
becomes
intcon (10)
downtosy
intcon (1)
dosy
ident (i)
becomes
ident (i)
minus
```

```
intcon (1)
semicolon
whilesy
ident (i)
gtr
intcon (0)
dosy
ident (i)
becomes
ident (i)
minus
intcon (1)
semicolon
repeatsy
ident (i)
becomes
ident (i)
plus
intcon (1)
semicolon
untilsy
ident (i)
eql
intcon (10)
semicolon
casesy
ident (i)
ofsy
intcon (1)
colon
ident (i)
becomes
intcon (100)
semicolon
intcon (2)
colon
ident (i)
becomes
intcon (200)
semicolon
endsy
semicolon
endsy
period
```

output-5-parser.txt


```

<program>
  <program-header>
    programsy
    ident (CommentTest)
    semicolon
  <declaration-part>
    <var-declaration>
      varsy
      <identifier-list>
        ident (x)
      colon
      <type>
        ident (integer)
      semicolon
      <identifier-list>
        ident (i)
      colon
      <type>
        ident (integer)
      semicolon
    <compound-statement>
      beginsy
      <statement-list>
        <assignment-statement>
          <variable>
            ident (x)
          becomes
          <expression>
            <simple-expression>
              <term>
                <factor>
                  intcon (5)
          semicolon
        <assignment-statement>
          <variable>
            ident (x)
          becomes
          <expression>
            <simple-expression>
              <term>
                <factor>
                  ident (x)
              <additive-operator>
                plus
              <term>
                <factor>

```

```

                                intcon (10)
semicolon
<assignment-statement>
  <variable>
    ident (x)
  becomes
  <expression>
    <simple-expression>
      <term>
        <factor>
          string (Not a { comment })
semicolon
<for-statement>
  forsy
  ident (i)
  becomes
  <expression>
    <simple-expression>
      <term>
        <factor>
          intcon (1)

  tosy
  <expression>
    <simple-expression>
      <term>
        <factor>
          intcon (10)

  dosy
  <assignment-statement>
    <variable>
      ident (i)
    becomes
    <expression>
      <simple-expression>
        <term>
          <factor>
            ident (i)
          <additive-operator>
            plus
          <term>
            <factor>
              intcon (1)
semicolon
<for-statement>
  forsy
  ident (i)

```

```

becomes
<expression>
  <simple-expression>
    <term>
      <factor>
        intcon (10)

downtosy
<expression>
  <simple-expression>
    <term>
      <factor>
        intcon (1)

dosy
<assignment-statement>
  <variable>
    ident (i)
  becomes
  <expression>
    <simple-expression>
      <term>
        <factor>
          ident (i)
        <additive-operator>
          minus
      <term>
        <factor>
          intcon (1)

semicolon
<while-statement>
  whilesy
  <expression>
    <simple-expression>
      <term>
        <factor>
          ident (i)

  gtr
  <simple-expression>
    <term>
      <factor>
        intcon (0)

dosy
<assignment-statement>
  <variable>
    ident (i)
  becomes
  <expression>

```

```

        <simple-expression>
            <term>
                <factor>
                    ident (i)
            <additive-operator>
                minus
            <term>
                <factor>
                    intcon (1)
semicolon
<repeat-statement>
    repeatsy
    <statement-list>
        <assignment-statement>
            <variable>
                ident (i)
            becomes
            <expression>
                <simple-expression>
                    <term>
                        <factor>
                            ident (i)
                    <additive-operator>
                        plus
                    <term>
                        <factor>
                            intcon (1)
                semicolon
            untilsy
            <expression>
                <simple-expression>
                    <term>
                        <factor>
                            ident (i)
                eql
                <simple-expression>
                    <term>
                        <factor>
                            intcon (10)
            semicolon
        <case-statement>
            casesy
            <expression>
                <simple-expression>
                    <term>
                        <factor>

```

```

                                ident (i)
ofsy
<case-list>
  <case-branch>
    <constant-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
              intcon (1)
    colon
    <assignment-statement>
      <variable>
        ident (i)
      becomes
      <expression>
        <simple-expression>
          <term>
            <factor>
              intcon (100)
  semicolon
  <case-branch>
    <constant-list>
      <expression>
        <simple-expression>
          <term>
            <factor>
              intcon (2)
    colon
    <assignment-statement>
      <variable>
        ident (i)
      becomes
      <expression>
        <simple-expression>
          <term>
            <factor>
              intcon (200)
  semicolon
endsy
semicolon
endsy
period

```

6. Test Case 6

Pengujian pada kasus program yang mendefinisikan tipe baru, penggunaan array, serta subprogram.

input-6.txt

```
program TypeArraySubprogram;

type
  TPoint == record
    x: integer;
    y: integer
  end;
  TNumbers == array[1..100] of integer;
  TMatrix == array[1..10] of array[1..10] of real;

var
  points: array[1..5] of TPoint;
  numbers: TNumbers;
  matrix: TMatrix;
  name: string;
  count: integer;

(* procedure PrintPoint(p: TPoint);
begin
  writeln('Point: (', p.x, ', ', p.y, ')');
end; *)

procedure InitializeArray(arr: TNumbers; size: integer);
var
  i: integer;
begin
  for i := 1 to size do
    arr[i] := i * 10;
  end;

function CalculateSum(arr: TNumbers; size: integer): integer;
var
  i: integer;
  total: integer;
begin
  total := 0;
  for i := 1 to size do
    total := total + arr[i];
  end;
  CalculateSum := total;
end;
```

```

procedure FillMatrix(m: TMatrix; rows: integer; cols: integer);
var
  i: integer;
  j: integer;
begin
  for i := 1 to rows do
    for j := 1 to cols do
      m[i][j] := i + j;
    end;
  end;

begin
  count := 5;

  { Initialize points }
  points[1].x := 1;
  points[1].y := 2;
  points[2].x := 3;
  points[2].y := 4;

  { PrintPoint(points[1]); }

  { Initialize and process numbers array }
  InitializeArray(numbers, 10);
  writeln('Sum = ', CalculateSum(numbers, 10));

  { Fill matrix }
  FillMatrix(matrix, 5, 5);

  { Test string }
  name := 'TestProgram';
  writeln('Program name: ', name);
end.

```

output-6-lexer.txt

```

programsy
ident (TypeArraySubprogram)
semicolon
typesy
ident (TPoint)
eq1
recordsy
ident (x)
colon
ident (integer)
semicolon
ident (y)

```

```
colon
ident (integer)
endsy
semicolon
ident (TNumbers)
eq1
arraysy
lbrack
intcon (1)
period
period
intcon (100)
rbrack
ofsy
ident (integer)
semicolon
ident (TMatrix)
eq1
arraysy
lbrack
intcon (1)
period
period
intcon (10)
rbrack
ofsy
arraysy
lbrack
intcon (1)
period
period
intcon (10)
rbrack
ofsy
ident (real)
semicolon
varsy
ident (points)
colon
arraysy
lbrack
intcon (1)
period
period
intcon (5)
rbrack
```



```
ofsy
ident (TPoint)
semicolon
ident (numbers)
colon
ident (TNumbers)
semicolon
ident (matrix)
colon
ident (TMatrix)
semicolon
ident (name)
colon
ident (string)
semicolon
ident (count)
colon
ident (integer)
semicolon
proceduresy
ident (InitializeArray)
lparent
ident (arr)
colon
ident (TNumbers)
semicolon
ident (size)
colon
ident (integer)
rparent
semicolon
varsy
ident (i)
colon
ident (integer)
semicolon
beginsy
forsy
ident (i)
becomes
intcon (1)
tosy
ident (size)
dosy
ident (arr)
lbrack
```

```
ident (i)
rbrack
becomes
ident (i)
times
intcon (10)
semicolon
endsy
semicolon
functionsy
ident (CalculateSum)
lparent
ident (arr)
colon
ident (TNumbers)
semicolon
ident (size)
colon
ident (integer)
rparent
colon
ident (integer)
semicolon
varsy
ident (i)
colon
ident (integer)
semicolon
ident (total)
colon
ident (integer)
semicolon
beginsy
ident (total)
becomes
intcon (0)
semicolon
forsy
ident (i)
becomes
intcon (1)
tosy
ident (size)
dosy
ident (total)
becomes
```

```
ident (total)
plus
ident (arr)
lbrack
ident (i)
rbrack
semicolon
ident (CalculateSum)
becomes
ident (total)
semicolon
endsy
semicolon
proceduresy
ident (FillMatrix)
lparent
ident (m)
colon
ident (TMatrix)
semicolon
ident (rows)
colon
ident (integer)
semicolon
ident (cols)
colon
ident (integer)
rparent
semicolon
varsy
ident (i)
colon
ident (integer)
semicolon
ident (j)
colon
ident (integer)
semicolon
beginsy
forsy
ident (i)
becomes
intcon (1)
tosy
ident (rows)
dosy
```

```
forsy
ident (j)
becomes
intcon (1)
tosy
ident (cols)
dosy
ident (m)
lbrack
ident (i)
rbrack
lbrack
ident (j)
rbrack
becomes
ident (i)
plus
ident (j)
semicolon
endsy
semicolon
beginsy
ident (count)
becomes
intcon (5)
semicolon
ident (points)
lbrack
intcon (1)
rbrack
period
ident (x)
becomes
intcon (1)
semicolon
ident (points)
lbrack
intcon (1)
rbrack
period
ident (y)
becomes
intcon (2)
semicolon
ident (points)
lbrack
```

```
intcon (2)
rbrack
period
ident (x)
becomes
intcon (3)
semicolon
ident (points)
lbrack
intcon (2)
rbrack
period
ident (y)
becomes
intcon (4)
semicolon
ident (InitializeArray)
lparent
ident (numbers)
comma
intcon (10)
rparent
semicolon
ident (writeln)
lparent
string (Sum = )
comma
ident (CalculateSum)
lparent
ident (numbers)
comma
intcon (10)
rparent
rparent
semicolon
ident (FillMatrix)
lparent
ident (matrix)
comma
intcon (5)
comma
intcon (5)
rparent
semicolon
ident (name)
becomes
```

```

string (TestProgram)
semicolon
ident (writeln)
lparent
string (Program name: )
comma
ident (name)
rparent
semicolon
endsy
period

```

output-6-parser.txt

```

<program>
  <program-header>
    programsy
    ident (TypeArraySubprogram)
    semicolon
  <declaration-part>
    <type-declaration>
      typesy
      ident (TPoint)
      eql
      <type>
        <record-type>
          recordsy
          <field-list>
            <field-part>
              <identifier-list>
                ident (x)
              colon
              <type>
                ident (integer)
            semicolon
            <field-part>
              <identifier-list>
                ident (y)
              colon
              <type>
                ident (integer)
          endsy
        semicolon
      ident (TNumbers)
      eql
      <type>
        <array-type>
          arraysy
          lbrack
          <range>
            <constant>
              intcon (1)
          period

```

```

        period
        <constant>
        intcon (100)
    rbrack
    ofsy
    <type>
    ident (integer)
semicolon
ident (TMatrix)
eql
<type>
    <array-type>
    arraysy
    lbrack
    <range>
    <constant>
    intcon (1)
    period
    period
    <constant>
    intcon (10)
    rbrack
    ofsy
    <type>
    <array-type>
    arraysy
    lbrack
    <range>
    <constant>
    intcon (1)
    period
    period
    <constant>
    intcon (10)
    rbrack
    ofsy
    <type>
    ident (real)

semicolon
<var-declaration>
varsy
<identifier-list>
    ident (points)
colon
<type>
    <array-type>
    arraysy
    lbrack
    <range>
    <constant>
    intcon (1)
    period
    period
    <constant>
    intcon (5)
    rbrack

```

```

        ofsy
        <type>
            ident (TPoint)
    semicolon
    <identifier-list>
        ident (numbers)
    colon
    <type>
        ident (TNumbers)
    semicolon
    <identifier-list>
        ident (matrix)
    colon
    <type>
        ident (TMatrix)
    semicolon
    <identifier-list>
        ident (name)
    colon
    <type>
        ident (string)
    semicolon
    <identifier-list>
        ident (count)
    colon
    <type>
        ident (integer)
    semicolon
<subprogram-declaration>
    <procedure-declaration>
        proceduresy
        ident (InitializeArray)
        <formal-parameter-list>
            lparent
            <parameter-group>
                <identifier-list>
                    ident (arr)
                colon
                ident (TNumbers)
            semicolon
            <parameter-group>
                <identifier-list>
                    ident (size)
                colon
                ident (integer)
            rparent
        semicolon
    block
        <declaration-part>
            <var-declaration>
                varsy
                <identifier-list>
                    ident (i)
                colon
                <type>
                    ident (integer)

```



```

        semicolon
    <compound-statement>
        beginsy
        <statement-list>
            <for-statement>
                forsy
                ident (i)
                becomes
                <expression>
                    <simple-expression>
                        <term>
                            <factor>
                                intcon (1)

                tosy
                <expression>
                    <simple-expression>
                        <term>
                            <factor>
                                ident (size)

                dosy
                <assignment-statement>
                    <variable>
                        ident (arr)
                        <component-variable>
                            lbrack
                            <index-list>
                                ident (i)
                            rbrack
                    becomes
                    <expression>
                        <simple-expression>
                            <term>
                                <factor>
                                    ident (i)
                                times
                                <factor>
                                    intcon (10)

            semicolon
        endsy
    semicolon
<subprogram-declaration>
    <function-declaration>
        functionsy
        ident (CalculateSum)
        <formal-parameter-list>
            lparent
            <parameter-group>
                <identifier-list>
                    ident (arr)
                colon
                ident (TNumbers)
            semicolon
            <parameter-group>
                <identifier-list>
                    ident (size)
                colon

```

```

        ident (integer)
    rparent
colon
ident (integer)
semicolon
block
    <declaration-part>
        <var-declaration>
            varsy
            <identifier-list>
                ident (i)
            colon
            <type>
                ident (integer)
            semicolon
            <identifier-list>
                ident (total)
            colon
            <type>
                ident (integer)
            semicolon
        <compound-statement>
            beginsy
            <statement-list>
                <assignment-statement>
                    <variable>
                        ident (total)
                    becomes
                    <expression>
                        <simple-expression>
                            <term>
                                <factor>
                                    intcon (0)
                semicolon
            <for-statement>
                forsy
                ident (i)
                becomes
                <expression>
                    <simple-expression>
                        <term>
                            <factor>
                                intcon (1)
                tosy
                <expression>
                    <simple-expression>
                        <term>
                            <factor>
                                ident (size)
            dosy
            <assignment-statement>
                <variable>
                    ident (total)
                becomes
                <expression>
                    <simple-expression>

```

```

                                <term>
                                    <factor>
                                        ident (total)
                                <additive-operator>
                                    plus
                                <term>
                                    <factor>
                                        ident (arr)
                                        <variable>
                                            <component-variable>
                                                lbrack
                                                <index-list>
                                                    ident (i)
                                                rbrack
                                semicolon
                                <assignment-statement>
                                    <variable>
                                        ident (CalculateSum)
                                    becomes
                                    <expression>
                                        <simple-expression>
                                            <term>
                                                <factor>
                                                    ident (total)
                                semicolon
                                endsy
                                semicolon
                                <subprogram-declaration>
                                    <procedure-declaration>
                                        proceduresy
                                        ident (FillMatrix)
                                        <formal-parameter-list>
                                            lparent
                                            <parameter-group>
                                                <identifier-list>
                                                    ident (m)
                                                colon
                                                ident (TMatrix)
                                            semicolon
                                            <parameter-group>
                                                <identifier-list>
                                                    ident (rows)
                                                colon
                                                ident (integer)
                                            semicolon
                                            <parameter-group>
                                                <identifier-list>
                                                    ident (cols)
                                                colon
                                                ident (integer)
                                            rparent
                                        semicolon
                                    block
                                        <declaration-part>
                                            <var-declaration>
                                                varsy

```

```

    <identifier-list>
        ident (i)
    colon
    <type>
        ident (integer)
    semicolon
    <identifier-list>
        ident (j)
    colon
    <type>
        ident (integer)
    semicolon
<compound-statement>
    beginsy
    <statement-list>
        <for-statement>
            forsy
            ident (i)
            becomes
            <expression>
                <simple-expression>
                    <term>
                        <factor>
                            intcon (1)
            tosy
            <expression>
                <simple-expression>
                    <term>
                        <factor>
                            ident (rows)
        dosy
        <for-statement>
            forsy
            ident (j)
            becomes
            <expression>
                <simple-expression>
                    <term>
                        <factor>
                            intcon (1)
            tosy
            <expression>
                <simple-expression>
                    <term>
                        <factor>
                            ident (cols)
        dosy
        <assignment-statement>
            <variable>
                ident (m)
                <component-variable>
                    lbrack
                    <index-list>
                        ident (i)
                    rbrack
                <component-variable>

```

```

                                lbrack
                                <index-list>
                                ident (j)
                                rbrack
                                becomes
                                <expression>
                                <simple-expression>
                                <term>
                                <factor>
                                ident (i)
                                <additive-operator>
                                plus
                                <term>
                                <factor>
                                ident (j)

                                semicolon
                                endsy
                                semicolon
                                <compound-statement>
                                beginsy
                                <statement-list>
                                <assignment-statement>
                                <variable>
                                ident (count)
                                becomes
                                <expression>
                                <simple-expression>
                                <term>
                                <factor>
                                intcon (5)
                                semicolon
                                <assignment-statement>
                                <variable>
                                ident (points)
                                <component-variable>
                                lbrack
                                <index-list>
                                intcon (1)
                                rbrack
                                <component-variable>
                                period
                                ident (x)
                                becomes
                                <expression>
                                <simple-expression>
                                <term>
                                <factor>
                                intcon (1)
                                semicolon
                                <assignment-statement>
                                <variable>
                                ident (points)
                                <component-variable>
                                lbrack
                                <index-list>
                                intcon (1)

```

```

        rbrack
        <component-variable>
        period
        ident (y)
    becomes
    <expression>
        <simple-expression>
            <term>
                <factor>
                    intcon (2)
semicolon
<assignment-statement>
    <variable>
        ident (points)
        <component-variable>
            lbrack
            <index-list>
                intcon (2)
            rbrack
        <component-variable>
            period
            ident (x)
    becomes
    <expression>
        <simple-expression>
            <term>
                <factor>
                    intcon (3)
semicolon
<assignment-statement>
    <variable>
        ident (points)
        <component-variable>
            lbrack
            <index-list>
                intcon (2)
            rbrack
        <component-variable>
            period
            ident (y)
    becomes
    <expression>
        <simple-expression>
            <term>
                <factor>
                    intcon (4)
semicolon
<procedure/function-call>
    ident (InitializeArray)
    lparent
    <parameter-list>
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        ident (numbers)

```

```

        comma
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        intcon (10)
rp
parent
semicolon
<procedure/function-call>
    ident (writeln)
    lparent
    <parameter-list>
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        string (Sum = )
        comma
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        ident (CalculateSum)
                        <procedure-call>
                            lparent
                            <parameter-list>
                                <expression>
                                    <simple-expression>
                                        <term>
                                            <factor>
                                                ident (numbers)
                                comma
                                <expression>
                                    <simple-expression>
                                        <term>
                                            <factor>
                                                intcon (10)
rp
parent
rp
parent
semicolon
<procedure/function-call>
    ident (FillMatrix)
    lparent
    <parameter-list>
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        ident (matrix)
        comma
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        intcon (5)
    comma

```

```

        <expression>
            <simple-expression>
                <term>
                    <factor>
                        intcon (5)
rpparent
semicolon
<assignment-statement>
    <variable>
        ident (name)
    becomes
    <expression>
        <simple-expression>
            <term>
                <factor>
                    string (TestProgram)
semicolon
<procedure/function-call>
    ident (writeln)
    lparent
    <parameter-list>
        <expression>
            <simple-expression>
                <term>
                    <factor>
                        string (Program name: )
    comma
    <expression>
        <simple-expression>
            <term>
                <factor>
                    ident (name)
rpparent
semicolon
endsy
period

```


Kesimpulan dan Saran

Kesimpulan

Pada pengerjaan Milestone 2 Tugas Besar TBFO ini, kami berhasil untuk mengimplementasikan parser untuk token hasil lexer pada grammar bahasa Arion yang sudah dikerjakan pada Milestone 1. Parser ini dibuat dengan menggunakan algoritma Recursive Descent Parser dan dapat mengenali seluruh struktur grammar yang didefinisikan sesuai spesifikasi tugas, mulai dari deklarasi, compound statement, hingga ekspresi bertingkat. Program ini juga sudah dilakukan pengujian pada beberapa test case untuk memastikan program sesuai dengan fungsionalnya, baik sesuai control flow, program kompleks, hingga penanganan syntax error. Pengembangan program parser secara modular dari hasil penyesuaian dari Milestone 1 berhasil diimplementasikan dan memudahkan pengerjaan tim dan pengintegrasian program.

Saran

Untuk pengembangan selanjutnya pada Milestone 3, harapannya core program yang sudah kami rapikan pada Milestone 2 ini dapat berguna untuk mempermudah pengembangan program untuk Abstract Syntax Tree yang lebih efisien dan berjalan tanpa banyak bug. Terlepas dari program yang sudah berhasil berjalan sebagaimana mestinya, tentunya masih banyak kekurangan yang dimiliki dari implementasi ini yang mungkin kedepannya dapat disesuaikan agar program lebih efisien. Oleh karena itu, kami sangat terbuka terhadap kritik dan saran dari asisten terkait tugas yang kami kerjakan agar kami dapat terus belajar dan meningkatkan kualitas program dan pengerjaan tugas menjadi lebih baik lagi.

Lampiran

1. Link Repository Github

<https://github.com/keirzka/AMN-Tubes-IF2224-2026.git>

2. Pembagian Tugas

Nama / NIM	Deskripsi Tugas	Persentase Kontribusi
13524071 Kalyca Nathania Benedicta Manullang	Membuat implementasi program bagian statement dan control flow	25%
	Mengerjakan laporan tugas bagian implementasi program	
13524073 Keisha Rizka Syofyani	Membuat implementasi core program parser dan program utama	25%
	Refactor dan menyesuaikan struktur file dan program Milestone 1 serta revisi program pada Milestone 1	
	Membuat inisialisasi laporan, quality controlling isi laporan, mengerjakan bagian kesimpulan dan saran, serta finishing readme dan submission.	
13524087 Muhammad Fakhry Zaki	Membuat implementasi program bagian expression parser dan operator	25%
	Membuat implementasi parser tree builder	
	Melakukan testing dan bug fixing	
	Mengerjakan laporan bagian testing program	
13524109 Helena Kristela Sarhawa	Membuat implementasi program bagian declaration parser dan menangani edge case parsing declaration	25%
	Mengerjakan laporan tugas bagian landasan teori	

